



# **Pemrograman Backend Lanjut (Praktikum)**

TI - C5

## **LAPORAN UJIAN TENGAH SEMESTER (RESTful API SIAKAD Mini)**

TEKNIK INFORMATIKA  
FAKULTAS VOKASI  
UNIVERSITAS AIRLANGGA

[434241070] | [Ocha Della Fitriani]  
[8 Oktober 2026]

## **KATA PENGANTAR**

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya laporan Ujian Tengah Semester mata kuliah Pemrograman Backend Lanjut (Praktikum) ini dapat saya selesaikan dengan baik. Di laporan ini saya mendokumentasikan pembuatan RESTful API SIAKAD Mini, yaitu layanan akademik sederhana untuk mengelola data mahasiswa, mata kuliah, dan Kartu Rencana Studi.

Laporan ini saya susun berdasarkan langkah-langkah yang benar-benar saya kerjakan, mulai dari persiapan lingkungan kerja, pembuatan database, penulisan kode, pengujian semua endpoint, sampai kendala yang saya temui selama pengerjaan. Materinya bersumber dari modul praktikum Pertemuan 1 sampai Pertemuan 5.

Saya mengucapkan terima kasih kepada dosen dan asisten praktikum Pemrograman Backend Lanjut atas bimbingan dan materi yang sudah diberikan selama setengah semester ini. Saya menyadari laporan ini masih punya kekurangan, jadi kritik dan saran yang membangun sangat saya harapkan.

Surabaya, 8 Oktober 2026

Ocha Della Fitriani

## DAFTAR ISI

|                                                                |           |
|----------------------------------------------------------------|-----------|
| KATA PENGANTAR                                                 | i         |
| DAFTAR ISI                                                     | ii        |
| DAFTAR TABEL                                                   | iv        |
| DAFTAR GAMBAR                                                  | v         |
| <b>BAB I PENDAHULUAN</b>                                       | <b>1</b>  |
| 1.1 Latar Belakang                                             | 1         |
| 1.2 Tujuan                                                     | 1         |
| 1.3 Ruang Lingkup                                              | 2         |
| 1.4 Lingkungan Pengerjaan                                      | 3         |
| 1.5 Struktur Proyek                                            | 3         |
| <b>BAB II LANGKAH Pengerjaan</b>                               | <b>5</b>  |
| 2.1 Langkah 1 Persiapan Lingkungan Kerja                       | 5         |
| 2.2 Langkah 2 Membuat Proyek dan Repositori GitHub             | 6         |
| 2.3 Langkah 3 Instalasi Dependency                             | 8         |
| 2.4 Langkah 4 Basis Data dan Konfigurasi Environment           | 9         |
| 2.5 Langkah 5 Migration Tabel                                  | 11        |
| 2.6 Langkah 6 Seeder Data Awal                                 | 14        |
| 2.7 Langkah 7 Autentikasi: Login dan Profil (Endpoint 1 dan 2) | 21        |
| 2.8 Langkah 8 Daftar Mata Kuliah (Endpoint 8)                  | 31        |
| 2.9 Langkah 9 CRUD Mahasiswa (Endpoint 3 sampai 7)             | 34        |
| 2.10 Langkah 10 Kartu Rencana Studi (Endpoint 9 dan 10)        | 42        |
| 2.11 Langkah 11 Dokumentasi dan Riwayat Commit                 | 51        |
| <b>BAB III PENGUJIAN</b>                                       | <b>53</b> |
| 3.1 Metode Pengujian                                           | 53        |
| 3.2 Pengujian Autentikasi                                      | 55        |
| 3.3 Pengujian Daftar Mata Kuliah                               | 59        |
| 3.4 Pengujian Data Mahasiswa                                   | 63        |
| 3.5 Pengujian Kartu Rencana Studi                              | 76        |
| 3.6 Rekapitulasi Hasil Pengujian                               | 79        |
| <b>BAB IV KENDALA SELAMA Pengerjaan</b>                        | <b>81</b> |

|                                                       |    |
|-------------------------------------------------------|----|
| 4.1 Request Login dari REST Client Selalu Dibalas 422 | 81 |
| <b>BAB V PENUTUP</b>                                  | 86 |
| 5.1 Kesimpulan                                        | 86 |
| 5.2 Tautan Repositori GitHub                          | 87 |

## **DAFTAR TABEL**

|           |                                            |    |
|-----------|--------------------------------------------|----|
| Tabel 1.1 | Daftar Endpoint SIAKAD Mini                | 2  |
| Tabel 1.2 | Perangkat Lunak dan Pustaka yang Digunakan | 3  |
| Tabel 1.3 | Struktur Folder dan Tanggung Jawabnya      | 4  |
| Tabel 2.1 | Riwayat Commit Bertahap di GitHub          | 52 |
| Tabel 3.1 | Akun yang Digunakan dalam Pengujian        | 55 |
| Tabel 3.2 | Rekapitulasi Hasil Pengujian Endpoint      | 79 |

## DAFTAR GAMBAR

|             |                                                                   |    |
|-------------|-------------------------------------------------------------------|----|
| Gambar 2.1  | Pemeriksaan Versi Tools dan Status Layanan PostgreSQL             | 5  |
| Gambar 2.2  | Pembuatan Folder Proyek, go mod init, dan git init                | 6  |
| Gambar 2.3  | Menghubungkan Remote, Membuat Struktur Folder, dan Commit Pertama | 7  |
| Gambar 2.4  | Struktur Folder Proyek setelah Commit Pertama                     | 8  |
| Gambar 2.5  | Isi File .gitignore                                               | 8  |
| Gambar 2.6  | Instalasi Pustaka dengan go get                                   | 9  |
| Gambar 2.7  | Pemeriksaan User Basis Data yang Aktif                            | 9  |
| Gambar 2.8  | Pembuatan Basis Data dan JWT Secret                               | 10 |
| Gambar 2.9  | Isi File .env.example                                             | 10 |
| Gambar 2.10 | Pemeriksaan git status dan Commit Konfigurasi                     | 11 |
| Gambar 2.11 | Empat File Migration Baru di Folder migrations                    | 11 |
| Gambar 2.12 | Migration Tabel users                                             | 12 |
| Gambar 2.13 | Migration Tabel students                                          | 12 |
| Gambar 2.14 | Migration Tabel courses                                           | 13 |
| Gambar 2.15 | Migration Tabel enrollments                                       | 13 |
| Gambar 2.16 | Menjalankan Migration dan Memeriksa Daftar Tabel                  | 14 |
| Gambar 2.17 | File Baru untuk Seeder                                            | 14 |
| Gambar 2.18 | Isi File config/env.go                                            | 15 |
| Gambar 2.19 | Isi File database/postgres.go                                     | 16 |
| Gambar 2.20 | Isi File helper/security.go                                       | 17 |
| Gambar 2.21 | Seeder Bagian Awal: Transaction, TRUNCATE, dan Akun Admin         | 18 |
| Gambar 2.22 | Seeder Bagian Data dan Perulangan 20 Mahasiswa                    | 19 |
| Gambar 2.23 | Seeder Bagian Mata Kuliah dan Commit Transaction                  | 20 |
| Gambar 2.24 | Menjalankan Seeder dan Memverifikasi Jumlah Data                  | 20 |
| Gambar 2.25 | File Baru untuk Server dan Autentikasi                            | 21 |
| Gambar 2.26 | Struktur Response Seragam pada helper/response.go                 | 22 |
| Gambar 2.27 | Fungsi Pembentuk Response pada helper/response.go                 | 22 |
| Gambar 2.28 | Pembuatan dan Pemeriksaan Token pada helper/jwt.go                | 23 |

|             |                                                                |    |
|-------------|----------------------------------------------------------------|----|
| Gambar 2.29 | Kondisi Akun Aktif pada app/repository/user_repository.go      | 24 |
| Gambar 2.30 | Aturan Validasi Login pada app/service/auth_rules.go           | 25 |
| Gambar 2.31 | Fungsi Login pada app/service/auth_service.go                  | 26 |
| Gambar 2.32 | Fungsi Me pada app/service/auth_service.go                     | 27 |
| Gambar 2.33 | Middleware RequireAuth pada middleware/auth.go                 | 28 |
| Gambar 2.34 | Middleware RequireRole dan LoginRateLimiter                    | 29 |
| Gambar 2.35 | Instance Fiber dan Error Handler pada config/app.go            | 30 |
| Gambar 2.36 | Server SIAKAD Mini API Berjalan di Port 3000                   | 31 |
| Gambar 2.37 | File Baru dan File yang Diubah untuk Endpoint Courses          | 31 |
| Gambar 2.38 | Query Daftar Mata Kuliah dengan Filter Dinamis                 | 32 |
| Gambar 2.39 | Perhitungan Sisa Kuota pada course_repository.go               | 33 |
| Gambar 2.40 | Validasi Query Parameter pada app/service/course_service.go    | 34 |
| Gambar 2.41 | File Baru dan File yang Diubah untuk CRUD Mahasiswa            | 35 |
| Gambar 2.42 | Business Rule Batas SKS pada student_rules.go                  | 35 |
| Gambar 2.43 | Validasi Tambah dan Ubah Mahasiswa pada student_rules.go       | 36 |
| Gambar 2.44 | Whitelist Sort dan Filter Daftar Mahasiswa                     | 37 |
| Gambar 2.45 | Pagination dengan LIMIT dan OFFSET                             | 38 |
| Gambar 2.46 | Tambah Mahasiswa dalam Satu Transaction                        | 39 |
| Gambar 2.47 | Ubah Data dan Soft Delete Mahasiswa                            | 40 |
| Gambar 2.48 | Pemeriksaan Kepemilikan Data pada Detail Mahasiswa             | 41 |
| Gambar 2.49 | Pendaftaran Route dan Penjaga Role pada route/route.go         | 42 |
| Gambar 2.50 | File Baru untuk Endpoint KRS pada Folder model dan service     | 43 |
| Gambar 2.51 | File enrollment_repository.go pada Folder repository           | 43 |
| Gambar 2.52 | Definisi Error dan SKSLimitError pada enrollment_repository.go | 44 |
| Gambar 2.53 | Awal Transaction dan Penguncian Baris Mahasiswa                | 45 |
| Gambar 2.54 | Penguncian Mata Kuliah, Cek Duplikasi, dan Cek Kuota           | 46 |
| Gambar 2.55 | Cek Batas SKS, Insert, dan Commit                              | 47 |
| Gambar 2.56 | Validasi Tahun Akademik pada enrollment_rules.go               | 48 |

|             |                                                                       |    |
|-------------|-----------------------------------------------------------------------|----|
| Gambar 2.57 | Pemetaan Error ke Status Code pada <code>enrollment_service.go</code> | 49 |
| Gambar 2.58 | Pembatalan KRS dengan Pemeriksaan Kepemilikan                         | 50 |
| Gambar 2.59 | Perakitan Repository, Service, dan Route pada <code>main.go</code>    | 51 |
| Gambar 2.60 | Riwayat Commit dan Status Akhir Repositori                            | 52 |
| Gambar 3.1  | Pemasangan Extension REST Client di VS Code                           | 53 |
| Gambar 3.2  | Isi File <code>requests.http</code> Bagian Endpoint Students          | 54 |
| Gambar 3.3  | Variabel Token Mahasiswa dan Admin pada <code>requests.http</code>    | 54 |
| Gambar 3.4  | Pengujian 1: Login Mahasiswa Berhasil (200)                           | 55 |
| Gambar 3.5  | Pengujian 2: Profil Mahasiswa yang Sedang Login (200)                 | 56 |
| Gambar 3.6  | Pengujian 3: Login Admin Berhasil (200)                               | 56 |
| Gambar 3.7  | Pengujian 4: Profil Admin (200)                                       | 57 |
| Gambar 3.8  | Pengujian 5: Login dengan Password Salah (401)                        | 57 |
| Gambar 3.9  | Pengujian 6: Validasi Login Gagal (422)                               | 58 |
| Gambar 3.10 | Pengujian 7: Profil tanpa Token (401)                                 | 58 |
| Gambar 3.11 | Pengujian 8: Profil dengan Token Palsu (401)                          | 59 |
| Gambar 3.12 | Pengujian 9: Daftar Seluruh Mata Kuliah (200)                         | 60 |
| Gambar 3.13 | Pengujian 10: Filter Semester dan Pencarian (200)                     | 61 |
| Gambar 3.14 | Pengujian 11: Mata Kuliah yang Kuotanya Belum Penuh (200)             | 62 |
| Gambar 3.15 | Pengujian 12: Query Parameter Tidak Valid (422)                       | 63 |
| Gambar 3.16 | Pengujian 13: Daftar Mata Kuliah tanpa Token (401)                    | 63 |
| Gambar 3.17 | Pengujian 14: Daftar Mahasiswa Bagian Awal (200)                      | 64 |
| Gambar 3.18 | Pengujian 14: Objek meta pada Daftar Mahasiswa                        | 65 |
| Gambar 3.19 | Pengujian 15: Pagination dan Urutan IPK Tertinggi (200)               | 66 |
| Gambar 3.20 | Pengujian 15: Objek meta dengan <code>per_page</code> 5               | 67 |
| Gambar 3.21 | Pengujian 16: Filter Prodi, Angkatan, dan Pencarian (200)             | 68 |
| Gambar 3.22 | Pengujian 17: Daftar Mahasiswa dengan Token Mahasiswa (403)           | 69 |
| Gambar 3.23 | Pengujian 18: Tambah Mahasiswa (201)                                  | 69 |
| Gambar 3.24 | Pengujian 19: Tambah Mahasiswa dengan NIM dan Email Ganda (422)       | 70 |

|             |                                                              |    |
|-------------|--------------------------------------------------------------|----|
| Gambar 3.25 | Pengujian 20: Tambah Mahasiswa dengan Data Tidak Valid (422) | 71 |
| Gambar 3.26 | Pengujian 21: Detail Mahasiswa oleh Admin (200)              | 72 |
| Gambar 3.27 | Pengujian 22: Mahasiswa Melihat Data Sendiri (200)           | 73 |
| Gambar 3.28 | Pengujian 23: Mahasiswa Melihat Data Orang Lain (403)        | 73 |
| Gambar 3.29 | Pengujian 24: Detail Mahasiswa yang Tidak Ada (404)          | 74 |
| Gambar 3.30 | Pengujian 25: Ubah Data Mahasiswa (200)                      | 74 |
| Gambar 3.31 | Pengujian 26: Mencoba Mengubah NIM (422)                     | 75 |
| Gambar 3.32 | Pengujian 27: Soft Delete Mahasiswa (204)                    | 75 |
| Gambar 3.33 | Pengujian 28: Menghapus Data yang Sudah Dihapus (404)        | 75 |
| Gambar 3.34 | Pengujian 29: Mengambil Mata Kuliah (201)                    | 76 |
| Gambar 3.35 | Pengujian 30: Mengambil Mata Kuliah yang Sama Dua Kali (409) | 76 |
| Gambar 3.36 | Pengujian 31: Mata Kuliah Tidak Ditemukan (422)              | 77 |
| Gambar 3.37 | Pengujian 32: Format Tahun Akademik Salah (422)              | 77 |
| Gambar 3.38 | Pengujian 33: Admin Mencoba Mengambil Mata Kuliah (403)      | 78 |
| Gambar 3.39 | Pengujian 34: KRS Terlihat pada Detail Mahasiswa (200)       | 78 |
| Gambar 3.40 | Pengujian 35: Membatalkan Mata Kuliah dari KRS (204)         | 79 |
| Gambar 3.41 | Pengujian 36: Membatalkan KRS yang Tidak Ada (404)           | 79 |
| Gambar 4.1  | Request Login Benar tetapi Dibalas 422                       | 81 |
| Gambar 4.2  | Log Debug Sementara pada Fungsi Login                        | 82 |
| Gambar 4.3  | Isi Body yang Diterima Server Ternyata Seluruh Isi File      | 83 |
| Gambar 4.4  | Seluruh Isi File Terseleksi saat Send Request Dijalankan     | 84 |
| Gambar 4.5  | Login Berhasil setelah Seleksi Dihilangkan                   | 85 |
| Gambar 4.6  | Variabel Token Otomatis Tidak Terbaca sehingga Dibalas 401   | 85 |

# BAB I PENDAHULUAN

## 1.1 Latar Belakang

Selama setengah semester ini, praktikum Pemrograman Backend Lanjut sudah membahas lima materi secara berurutan, yaitu setup lingkungan dan sintaks Go dengan Fiber (Pertemuan 1), REST API dan HTTP (Pertemuan 2), database PostgreSQL dengan pola repository (Pertemuan 3), Clean Architecture (Pertemuan 4), serta autentikasi dengan JWT dan bcrypt (Pertemuan 5). Masing-masing materi itu saya kerjakan terpisah di setiap pertemuan, tapi belum pernah saya gabungkan jadi satu aplikasi yang utuh.

Nah, di UTS ini semua materi tersebut dirangkai dalam satu studi kasus, yaitu *RESTful API back end* untuk **SIKAD Mini**. Aplikasinya berupa layanan akademik sederhana untuk mengelola data mahasiswa, mata kuliah, dan Kartu Rencana Studi (KRS). Proyeknya saya mulai dari folder kosong, tapi pola kodenya tetap mengikuti struktur baku dari Modul Pertemuan 4, jadi setiap file punya tugas yang jelas.

Ada beberapa kebutuhan UTS yang belum dibahas di Modul 1 sampai 5, yaitu pembatasan akses berdasarkan role (admin dan mahasiswa), transaction dengan row locking, soft delete, dan seeder. Bagian-bagian ini saya kerjakan dengan mengembangkan pola yang sudah ada. Contohnya, middleware `RequireRole` saya buat di atas `RequireAuth` dari Modul 5.

## 1.2 Tujuan

Tujuan saya mengerjakan UTS ini adalah sebagai berikut.

1. Membangun RESTful API SIKAD Mini menggunakan Go, framework Fiber v2, dan database PostgreSQL sesuai ketentuan teknis UTS.
2. Membuat migration dan seeder yang menghasilkan minimal 1 admin, 20 mahasiswa, dan 10 mata kuliah.

3. Mengamankan semua endpoint (kecuali login) dengan token JWT, menyimpan password dalam bentuk hash, dan membatasi percobaan login yang gagal.
4. Menerapkan empat business rule KRS secara konsisten, termasuk pengecekan kuota dengan transaction dan row locking.
5. Mendokumentasikan proses pengerjaan, pengujian, dan kendala yang saya hadapi, lengkap dengan riwayat commit bertahap di GitHub.

### 1.3 Ruang Lingkup

API ini dipakai oleh dua peran. **Admin** bertugas mengelola data mahasiswa, sedangkan **mahasiswa** bisa melihat profilnya sendiri serta mengambil dan membatalkan mata kuliah di KRS miliknya. Semua fungsi itu dilayani oleh sepuluh endpoint yang saya rangkum di Tabel 1.1.

**Tabel 1.1** *Daftar Endpoint SIAKAD Mini*

| No | Method | Endpoint                 | Akses                           | Sukses |
|----|--------|--------------------------|---------------------------------|--------|
| 1  | POST   | /api/v1/auth/login       | Publik                          | 200    |
| 2  | GET    | /api/v1/auth/me          | Semua role                      | 200    |
| 3  | GET    | /api/v1/students         | Admin                           | 200    |
| 4  | POST   | /api/v1/students         | Admin                           | 201    |
| 5  | GET    | /api/v1/students/{id}    | Admin, mahasiswa (data sendiri) | 200    |
| 6  | PUT    | /api/v1/students/{id}    | Admin                           | 200    |
| 7  | DELETE | /api/v1/students/{id}    | Admin                           | 204    |
| 8  | GET    | /api/v1/courses          | Semua role                      | 200    |
| 9  | POST   | /api/v1/enrollments      | Mahasiswa                       | 201    |
| 10 | DELETE | /api/v1/enrollments/{id} | Mahasiswa (milik sendiri)       | 204    |

Selain sepuluh endpoint itu, API juga wajib menjalankan empat business rule berikut.

1. Batas SKS per semester ditentukan oleh IPK terakhir:  $IPK \geq 3,00$  maksimal 24 SKS;  $IPK \ 2,50-2,99$  maksimal 21 SKS;  $IPK < 2,50$  maksimal 18 SKS.

2. Mahasiswa tidak boleh mengambil mata kuliah yang sama dua kali di tahun akademik yang sama.
3. Mata kuliah yang kuotanya sudah penuh tidak bisa diambil.
4. Mahasiswa hanya boleh mengakses dan mengubah KRS miliknya sendiri.

#### 1.4 Lingkungan Pengerjaan

Saya mengerjakan UTS ini di MacBook Air dengan chip Apple Silicon. Software dan library yang saya pakai ada di Tabel 1.2. Semua library Go-nya sama dengan yang dipakai di modul praktikum, jadi tidak ada teknologi baru di luar materi perkuliahan.

**Tabel 1.2** *Perangkat Lunak dan Pustaka yang Digunakan*

| Komponen              | Versi                 | Kegunaan                                        |
|-----------------------|-----------------------|-------------------------------------------------|
| Go                    | 1.26.6 (darwin/arm64) | Bahasa pemrograman                              |
| Fiber                 | v2.52.15              | Framework web (sesuai Modul 1)                  |
| PostgreSQL            | 16.11 (Homebrew)      | Basis data                                      |
| pgx                   | v5.11.0               | Driver PostgreSQL dan connection pool (Modul 3) |
| golang-jwt            | v5.3.1                | Pembuatan dan verifikasi token JWT (Modul 5)    |
| x/crypto/bcrypt       | v0.57.0               | Hashing password (Modul 5)                      |
| godotenv              | v1.5.1                | Membaca konfigurasi dari file .env (Modul 3)    |
| Git                   | 2.53.0                | Version control, repositori di GitHub           |
| VS Code + REST Client | REST Client 0.25.1    | Editor dan pengujian endpoint                   |

#### 1.5 Struktur Proyek

Struktur folder proyek saya ikuti dari struktur baku di Modul Pertemuan 4, yang membagi kode ke dalam empat lapisan Clean Architecture. Aturan paling pentingnya ada di arah ketergantungan: repository tidak boleh tahu soal fiber.Ctx, model tidak boleh bergantung ke apa pun, dan semua query SQL hanya boleh ada di folder repository. Pembagian tugas tiap folder bisa dilihat di Tabel 1.3.

**Tabel 1.3** *Struktur Folder dan Tanggung Jawabnya*

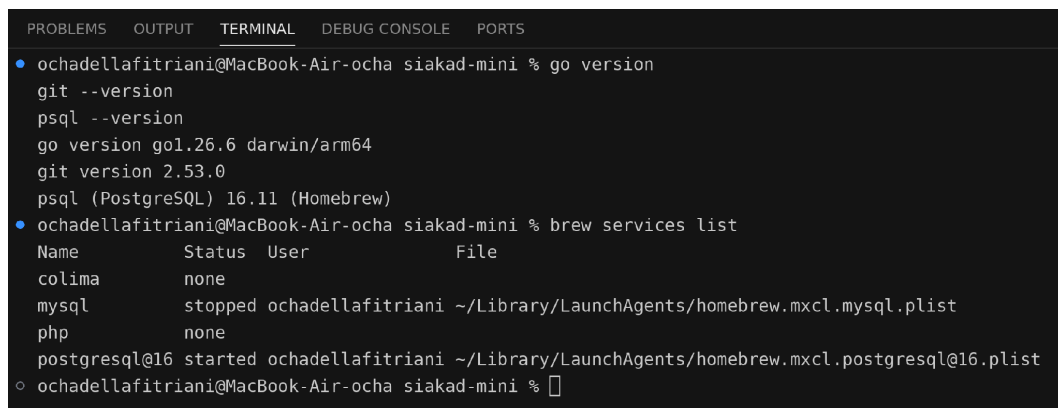
| Folder         | Isi                                           | Lapisan              |
|----------------|-----------------------------------------------|----------------------|
| app/model      | Struct entitas, request, dan response         | Entities             |
| app/repository | Query ke PostgreSQL (satu-satunya tempat SQL) | Interface Adapters   |
| app/service    | Business rules dan penerima fiber.Ctx         | Use Cases            |
| config         | Pembacaan .env, instance Fiber, error handler | Frameworks & Drivers |
| database       | Koneksi dan connection pool PostgreSQL        | Frameworks & Drivers |
| helper         | Format response, validasi, JWT, hashing       | Interface Adapters   |
| middleware     | RequireAuth, RequireRole, rate limiter        | Frameworks & Drivers |
| route          | Pendaftaran seluruh endpoint                  | Frameworks & Drivers |
| migrations     | File SQL pembuatan tabel                      | Frameworks & Drivers |
| cmd/seed       | Program seeder data awal                      | Frameworks & Drivers |

## BAB II LANGKAH Pengerjaan

Di bab ini saya jelaskan langkah-langkah pengerjaan secara urut, mulai dari folder kosong sampai sepuluh endpoint bisa berjalan. Setiap langkah saya lengkapi dengan perintah terminal atau kode yang saya tulis, beserta alasan kenapa langkah itu perlu. Sebagai catatan, file yang baru dibuat ditandai hijau dengan huruf U (untracked) di panel Explorer VS Code, sedangkan file yang diubah ditandai kuning dengan huruf M (modified). Setiap langkah saya tutup dengan satu commit, supaya riwayat pengerjaan di GitHub tersusun bertahap.

### 2.1 Langkah 1 Persiapan Lingkungan Kerja

Sebelum mulai ngoding, saya cek dulu lingkungan kerjanya lewat terminal VS Code. Saya jalankan tiga perintah versi untuk memastikan Go, Git, dan PostgreSQL sudah terpasang. Selain itu, saya juga menjalankan brew services list, karena PostgreSQL yang sudah terinstal belum tentu sedang jalan. Yang saya cari di sini adalah status postgresql@16 harus **started**.



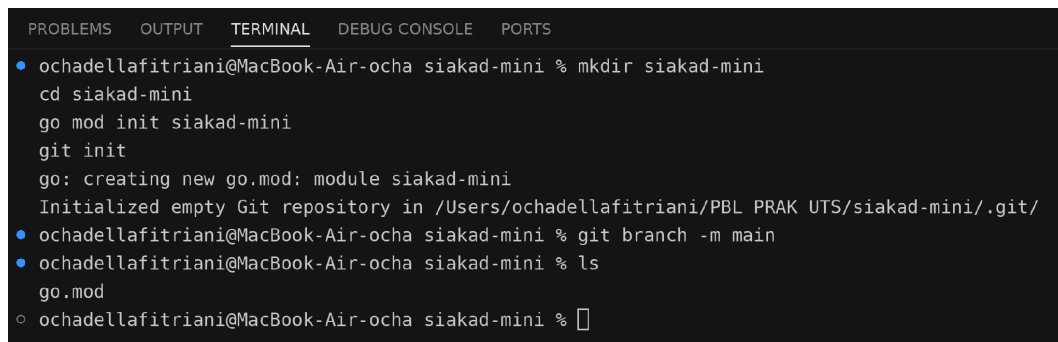
```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
ochadellafitriani@MacBook-Air-ocha siacad-mini % go version
git --version
psql --version
go version go1.26.6 darwin/arm64
git version 2.53.0
psql (PostgreSQL) 16.11 (Homebrew)
ochadellafitriani@MacBook-Air-ocha siacad-mini % brew services list
Name      Status  User              File
colima     none
mysql      stopped ochadellafitriani ~/Library/LaunchAgents/homebrew.mxcl.mysql.plist
php        none
postgresql@16 started ochadellafitriani ~/Library/LaunchAgents/homebrew.mxcl.postgresql@16.plist
ochadellafitriani@MacBook-Air-ocha siacad-mini %
```

**Gambar 2.1** Pemeriksaan Versi Tools dan Status Layanan PostgreSQL

Hasilnya, Go 1.26.6, Git 2.53.0, dan PostgreSQL 16.11 sudah tersedia, dan layanan PostgreSQL juga sudah berjalan. Ada satu hal yang saya catat dari awal: PostgreSQL yang diinstal lewat Homebrew tidak punya user postgres seperti contoh di modul. Username database-nya sama dengan username macOS saya, yaitu ochadellafitriani. Jadi di langkah-langkah berikutnya, perintah psql saya jalankan tanpa opsi -U postgres.

## 2.2 Langkah 2 Membuat Proyek dan Repositori GitHub

Saya membuat folder baru bernama `siakad-mini` di dalam folder `PBL PRAK UTS`, terpisah dari proyek latihan-fiber milik modul. Setelah itu saya jalankan `go mod init` untuk membuat file `go.mod`, yang isinya nama modul dan daftar dependensi, lalu `git init` untuk menyiapkan repositori lokal. Nama branch-nya saya ganti dari `master` menjadi `main`, supaya sama dengan branch bawaan GitHub.



```
PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE  PORTS
• ochadellafitriani@MacBook-Air-ocha siakad-mini % mkdir siakad-mini
  cd siakad-mini
  go mod init siakad-mini
  git init
  go: creating new go.mod: module siakad-mini
  Initialized empty Git repository in /Users/ochadellafitriani/PBL PRAK UTS/siakad-mini/.git/
• ochadellafitriani@MacBook-Air-ocha siakad-mini % git branch -m main
• ochadellafitriani@MacBook-Air-ocha siakad-mini % ls
  go.mod
○ ochadellafitriani@MacBook-Air-ocha siakad-mini %
```

**Gambar 2.2** Pembuatan Folder Proyek, `go mod init`, dan `git init`

Berikutnya saya membuat repositori kosong bernama `siakad-mini_UTSPBL` di GitHub, lalu menghubungkannya dengan `git remote add origin`. Semua folder struktur baku saya buat sekaligus dengan satu perintah `mkdir -p`. Karena Git tidak menyimpan folder kosong, setiap folder saya isi file `.gitkeep`. Saya juga membuat file `.gitignore` supaya file `.env` (yang berisi password database dan JWT secret), folder `logs`, dan file `.DS_Store` tidak ikut ter-upload. Sebelum commit pertama, saya atur dulu identitas Git dengan `git config --global user.name` dan `user.email`.

```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS

● otheadlafitriani@MacBook-Air-ocha siakad-mini % git remote add origin https://github.com/ochadella/siakad-mini_UTSPBL.git
git remote -v
origin https://github.com/ochadella/siakad-mini_UTSPBL.git (fetch)
origin https://github.com/ochadella/siakad-mini_UTSPBL.git (push)
● otheadlafitriani@MacBook-Air-ocha siakad-mini % mkdir -p app/model app/repository app/service config database helper middleware route migrations cmd/seed logs
● otheadlafitriani@MacBook-Air-ocha siakad-mini % printf ".env\nlogs/\n.DS_Store\n" > .gitignore
● otheadlafitriani@MacBook-Air-ocha siakad-mini % touch app/model/.gitkeep app/repository/.gitkeep app/service/.gitkeep config/.gitkeep database/.gitkeep helper/.gitkeep midd
eeep route/.gitkeep migrations/.gitkeep cmd/seed/.gitkeep
● otheadlafitriani@MacBook-Air-ocha siakad-mini % git add .
git status
On branch main

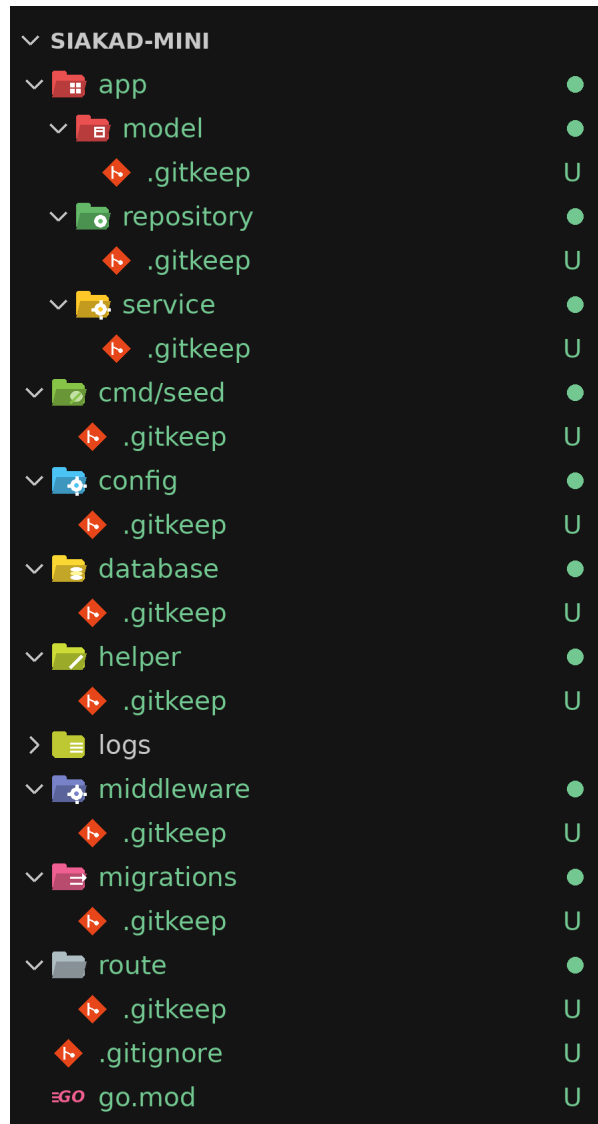
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   app/model/.gitkeep
    new file:   app/repository/.gitkeep
    new file:   app/service/.gitkeep
    new file:   cmd/seed/.gitkeep
    new file:   config/.gitkeep
    new file:   database/.gitkeep
    new file:   go.mod
    new file:   helper/.gitkeep
    new file:   middleware/.gitkeep
    new file:   migrations/.gitkeep
    new file:   route/.gitkeep

● otheadlafitriani@MacBook-Air-ocha siakad-mini % git config --global user.name "Ocha Della Fitriani"
git config --global user.email "ochadella66@gmail.com"
● otheadlafitriani@MacBook-Air-ocha siakad-mini % git commit -m "init: struktur folder proyek SIAKAD Mini"
git push -u origin main
[main (root-commit) ffa6cd] init: struktur folder proyek SIAKAD Mini
12 files changed, 6 insertions(+)
create mode 100644 .gitignore
create mode 100644 app/model/.gitkeep
create mode 100644 app/repository/.gitkeep
create mode 100644 app/service/.gitkeep
create mode 100644 cmd/seed/.gitkeep
create mode 100644 config/.gitkeep
create mode 100644 database/.gitkeep
create mode 100644 go.mod
create mode 100644 helper/.gitkeep
create mode 100644 middleware/.gitkeep
create mode 100644 migrations/.gitkeep
create mode 100644 route/.gitkeep
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (8/8), 640 bytes | 640.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ochadella/siakad-mini_UTSPBL.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
● otheadlafitriani@MacBook-Air-ocha siakad-mini %
```

**Gambar 2.3** *Menghubungkan Remote, Membuat Struktur Folder, dan Commit Pertama*

Di Gambar 2.3, hasil git status saya cek dulu sebelum commit. Yang masuk hanya .gitignore, go.mod, dan file-file .gitkeep, tanpa file .env. Setelah itu commit pertama berhasil di-push, ditandai dengan balasan GitHub **[new branch] main -> main**. Struktur folder yang terbentuk bisa dilihat di Gambar 2.4.



**Gambar 2.4** Struktur Folder Proyek setelah Commit Pertama

```
.gitignore
1 .env
2 logs/
3 .DS_Store
```

**Gambar 2.5** Isi File .gitignore

### 2.3 Langkah 3 Instalasi Dependency

Di langkah ini saya memasang lima library dengan go get. Library-nya sama persis dengan yang dipakai di Modul 1 sampai 5: Fiber untuk framework web,

godotenv untuk membaca file .env, pgx sebagai driver PostgreSQL, golang-jwt untuk token, dan bcrypt untuk hashing password. Tulisan added dan upgraded di terminal artinya Go ikut memasang library pendukung yang dibutuhkan library utama, sehingga file go.sum otomatis terbentuk.

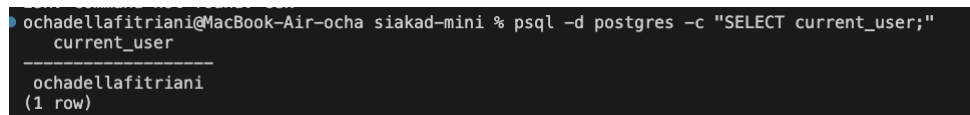


```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
ochadellafitriani@MacBook-Air-ocha siacad-mini % go get github.com/gofiber/fiber/v2
go get github.com/joho/godotenv
go get github.com/jackc/pgx/v5
go get github.com/golang-jwt/jwt/v5
go get golang.org/x/crypto/bcrypt
go: added github.com/andybalholm/brotli v1.1.0
go: added github.com/gofiber/fiber/v2 v2.52.15
go: added github.com/google/uuid v1.6.0
go: added github.com/klauspost/compress v1.17.9
go: added github.com/mattn/go-colorable v0.1.13
go: added github.com/mattn/go-isatty v0.0.20
go: added github.com/valyala/fasthttp v1.51.0
go: added github.com/joho/godotenv v1.5.1
go: added github.com/jackc/pgx/v5 v5.11.0
go: added github.com/golang-jwt/jwt/v5 v5.3.1
go: upgraded golang.org/x/crypto v0.14.0 => v0.57.0
ochadellafitriani@MacBook-Air-ocha siacad-mini %
```

**Gambar 2.6** Instalasi Pustaka dengan go get

## 2.4 Langkah 4 Basis Data dan Konfigurasi Environment

Langkah ini saya mulai dengan memastikan username database yang aktif. Dari query SELECT current\_user di Gambar 2.7, hasilnya **ochadellafitriani**, sesuai dugaan saya di Langkah 1. Username inilah yang nanti saya isi di variabel DB\_USER.



```
ochadellafitriani@MacBook-Air-ocha siacad-mini % psql -d postgres -c "SELECT current_user;"
current_user
-----
ochadellafitriani
(1 row)
```

**Gambar 2.7** Pemeriksaan User Basis Data yang Aktif

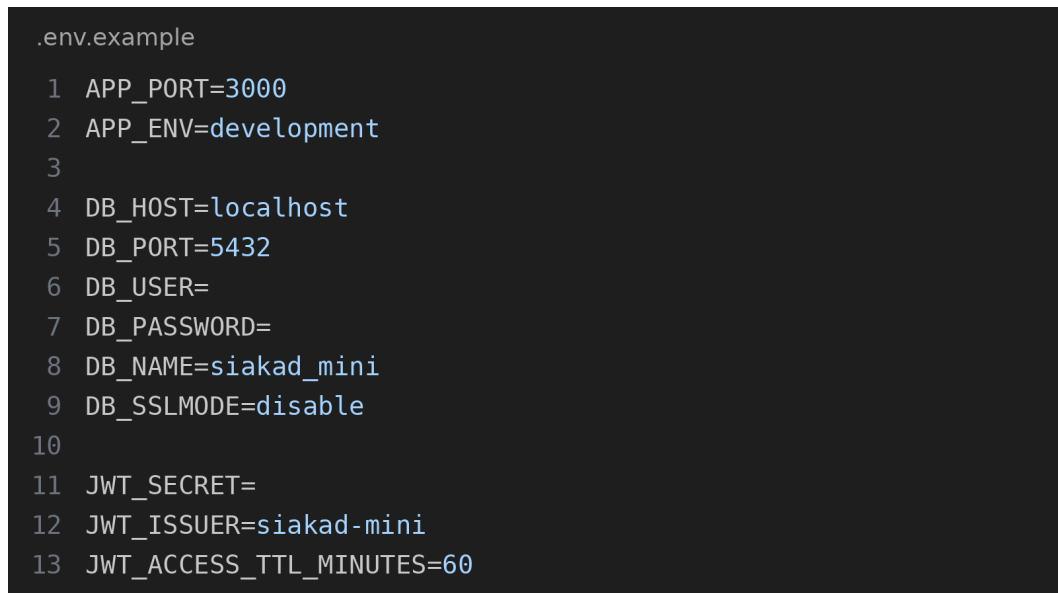
Setelah itu saya membuat database siacad\_mini dengan CREATE DATABASE, lalu membuat JWT secret menggunakan openssl rand -hex 32. Secret-nya sengaja dibuat acak sepanjang 32 byte, karena siapa pun yang tahu secret ini bisa membuat token atas nama siapa saja, termasuk admin.



```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
• ochadellafitriani@MacBook-Air-ocha siakad-mini % psql -d postgres -c "CREATE DATABASE siakad_mini;"
CREATE DATABASE
• ochadellafitriani@MacBook-Air-ocha siakad-mini % openssl rand -hex 32
1566909ab8b1859dc0ed3fbd9578714f2544616269781f3fc8585b0dd4062cea
• ochadellafitriani@MacBook-Air-ocha siakad-mini %
```

**Gambar 2.8** Pembuatan Basis Data dan JWT Secret

Buat file `.env` di folder utama proyek (sejajar dengan `go.mod`) yang berisi konfigurasi asli, lalu buat juga file `.env.example` dengan isi yang sama tapi nilai rahasianya dikosongkan. File `.env.example` inilah yang di-commit sebagai contoh untuk orang lain yang ingin menjalankan proyek ini. File `.env` yang asli tetap tinggal di laptop, karena sudah masuk `.gitignore`.



```
.env.example
1 APP_PORT=3000
2 APP_ENV=development
3
4 DB_HOST=localhost
5 DB_PORT=5432
6 DB_USER=
7 DB_PASSWORD=
8 DB_NAME=siakad_mini
9 DB_SSLMODE=disable
10
11 JWT_SECRET=
12 JWT_ISSUER=siakad-mini
13 JWT_ACCESS_TTL_MINUTES=60
```

**Gambar 2.9** Isi File `.env.example`

Sebelum commit, git status saya cek lagi. Isinya hanya `.env.example`, `go.mod`, dan `go.sum`, jadi file `.env` dipastikan tidak ikut ter-commit.

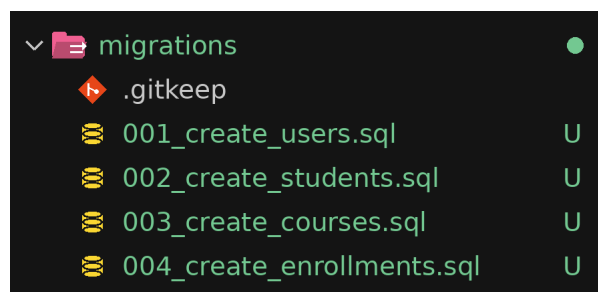
```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
• ochadellafitriani@MacBook-Air-ocha siacad-mini % git add .
git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .env.example
    modified:   go.mod
    new file:   go.sum
• ochadellafitriani@MacBook-Air-ocha siacad-mini % git commit -m "chore: install dependencies dan konfigurasi env"
git push
[main 069ba74] chore: install dependencies dan konfigurasi env
3 files changed, 89 insertions(+)
To https://github.com/ochadella/siacad-mini_UTSPBL.git
ffaf6cd..069ba74  main -> main
• ochadellafitriani@MacBook-Air-ocha siacad-mini %
```

**Gambar 2.10** *Pemeriksaan git status dan Commit Konfigurasi*

## 2.5 Langkah 5 Migration Tabel

Struktur tabel saya tulis sebagai file SQL di folder migrations, mengikuti prinsip di Modul 3 bahwa struktur tabel itu juga kode dan harus ikut tersimpan di repositori. Buat empat file migration di folder migrations/ dengan nomor urut di depan namanya. Nomor ini penting, karena tabel di file berikutnya bergantung ke tabel sebelumnya lewat FOREIGN KEY.



**Gambar 2.11** *Empat File Migration Baru di Folder migrations*

Tabel users menyimpan akun login. Kolom role saya batasi dengan **CHECK (role IN ('admin', 'mahasiswa'))**, jadi tidak mungkin ada akun dengan role selain dua itu. Kolom email saya beri UNIQUE karena email dipakai sebagai identitas login.

```
migrations > 001_create_users.sql
1 CREATE TABLE IF NOT EXISTS users (
2   id          SERIAL PRIMARY KEY,
3   email       VARCHAR(255) NOT NULL UNIQUE,
4   password    VARCHAR(255) NOT NULL,
5   role        VARCHAR(20)  NOT NULL CHECK (role IN ('admin', 'mahasiswa')),
6   created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
7   updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
8 );
```

**Gambar 2.12** *Migration Tabel users*

Di tabel students, kolom user\_id saya beri **UNIQUE** sekaligus REFERENCES users(id). Gabungan keduanya membentuk relasi satu-ke-satu, artinya satu akun hanya bisa dimiliki satu data mahasiswa. Kolom deleted\_at saya siapkan untuk soft delete, yaitu data tidak benar-benar dihapus, hanya ditandai waktu penghapusannya. Batas nilai IPK (0 sampai 4) dan angkatan (4 digit) juga saya pasang langsung di database, karena aturan di database lebih kuat daripada pengecekan di aplikasi.

```
migrations > 002_create_students.sql
1 CREATE TABLE IF NOT EXISTS students (
2   id          SERIAL PRIMARY KEY,
3   user_id     INT          NOT NULL UNIQUE REFERENCES users(id) ON DELETE CASCADE,
4   nim         CHAR(12)     NOT NULL UNIQUE,
5   nama        VARCHAR(100) NOT NULL,
6   prodi       VARCHAR(100) NOT NULL,
7   angkatan    INT          NOT NULL CHECK (angkatan BETWEEN 1000 AND 9999),
8   ipk_terakhir NUMERIC(3,2) NOT NULL DEFAULT 0 CHECK (ipk_terakhir BETWEEN 0 AND 4),
9   created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
10  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
11  deleted_at  TIMESTAMPTZ NULL
12 );
13
14 CREATE INDEX IF NOT EXISTS idx_students_deleted_at ON students(deleted_at);
```

**Gambar 2.13** *Migration Tabel students*

```
migrations > 003_create_courses.sql
1 CREATE TABLE IF NOT EXISTS courses (
2   id          SERIAL PRIMARY KEY,
3   kode_mk     VARCHAR(20) NOT NULL UNIQUE,
4   nama_mk     VARCHAR(150) NOT NULL,
5   sks         INT         NOT NULL CHECK (sks > 0),
6   semester   INT         NOT NULL CHECK (semester BETWEEN 1 AND 8),
7   kuota      INT         NOT NULL CHECK (kuota >= 0),
8   created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
9   updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
10 );
```

**Gambar 2.14** *Migration Tabel courses*

Tabel enrollments menyimpan isi KRS. Bagian terpentingnya adalah **CONSTRAINT uq\_enrollment UNIQUE (student\_id, course\_id, tahun\_akademik)**. Constraint ini jadi pertahanan terakhir untuk business rule nomor 2: walaupun aplikasi lupa mengecek, database tetap menolak mahasiswa yang mengambil mata kuliah yang sama dua kali di tahun akademik yang sama.

```
migrations > 004_create_enrollments.sql
1 CREATE TABLE IF NOT EXISTS enrollments (
2   id          SERIAL PRIMARY KEY,
3   student_id  INT         NOT NULL REFERENCES students(id) ON DELETE CASCADE,
4   course_id   INT         NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
5   tahun_akademik VARCHAR(20) NOT NULL,
6   created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
7   CONSTRAINT uq_enrollment UNIQUE (student_id, course_id, tahun_akademik)
8 );
9
10 CREATE INDEX IF NOT EXISTS idx_enrollments_course ON enrollments(course_id);
```

**Gambar 2.15** *Migration Tabel enrollments*

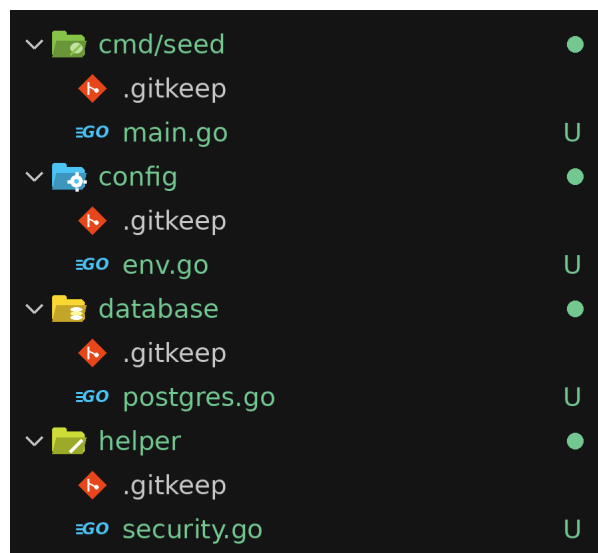
Keempat migration saya jalankan berurutan dengan psql -f, lalu hasilnya saya cek pakai perintah \dt. Hasilnya, empat tabel berhasil dibuat dengan pemilik ochadellafitriani.

```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
• ochadellafitriani@MacBook-Air-ocha siakad-mini % psql -d siakad_mini -f migrations/001_create_users.sql
psql -d siakad_mini -f migrations/002_create_students.sql
psql -d siakad_mini -f migrations/003_create_courses.sql
psql -d siakad_mini -f migrations/004_create_enrollments.sql
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE TABLE
CREATE TABLE
CREATE INDEX
• ochadellafitriani@MacBook-Air-ocha siakad-mini % psql -d siakad_mini -c "\dt"
List of relations
 Schema | Name      | Type  | Owner
-----+-----+-----+-----
 public | courses   | table | ochadellafitriani
 public | enrollments | table | ochadellafitriani
 public | students  | table | ochadellafitriani
 public | users     | table | ochadellafitriani
(4 rows)
○ ochadellafitriani@MacBook-Air-ocha siakad-mini %
```

**Gambar 2.16** Menjalankan Migration dan Memeriksa Daftar Tabel

## 2.6 Langkah 6 Seeder Data Awal

Seeder butuh tiga file pendukung yang nantinya juga dipakai aplikasi utama, yaitu pembaca file .env, koneksi database, dan fungsi hashing password. Buat file env.go di folder config/, postgres.go di folder database/, security.go di folder helper/, dan main.go di folder cmd/seed/.



**Gambar 2.17** File Baru untuk Seeder

File `config/env.go` membungkus library `godotenv`. Fungsi `GetEnv` akan mengembalikan nilai cadangan (fallback) kalau variabelnya kosong, jadi aplikasi tetap bisa jalan dengan nilai bawaan yang masuk akal.

```
config > env.go
1 package config
2
3 import (
4     "log"
5     "os"
6     "strconv"
7
8     "github.com/joho/godotenv"
9 )
10
11 // LoadEnv memuat variabel dari berkas .env ke environment proses.
12 func LoadEnv() {
13     if err := godotenv.Load(); err != nil {
14         log.Println("peringatan: berkas .env tidak ditemukan, memakai environment sistem")
15     }
16 }
17
18 // GetEnv membaca variabel environment, atau mengembalikan fallback bila kosong.
19 func GetEnv(key, fallback string) string {
20     if v, ok := os.LookupEnv(key); ok && v != "" {
21         return v
22     }
23     return fallback
24 }
25
26 // GetEnvInt sama seperti GetEnv, tetapi hasilnya berupa angka.
27 func GetEnvInt(key string, fallback int) int {
28     v := GetEnv(key, "")
29     if v == "" {
30         return fallback
31     }
32     n, err := strconv.Atoi(v)
33     if err != nil {
34         return fallback
35     }
36     return n
37 }
```

**Gambar 2.18** Isi File `config/env.go`

File `database/postgres.go` membuat connection pool dengan `pgxpool`. Alamat koneksinya saya susun pakai `url.URL` dan `url.UserPassword`, bukan dengan menggabungkan string manual, supaya password yang kosong atau mengandung karakter khusus tetap terbaca dengan benar. Di akhir fungsi ada Ping dengan batas waktu 5 detik, jadi kalau koneksi bermasalah, errornya langsung

ketahuan saat aplikasi dinyalakan, bukan baru ketahuan saat request pertama masuk.

```
database > postgres.go
1 package database
2
3 import (
4     "context"
5     "fmt"
6     "net/url"
7     "time"
8
9     "github.com/jackc/pgx/v5/pgxpool"
10
11     "siakad-mini/config"
12 )
13
14 // NewPool membuat connection pool ke PostgreSQL berdasarkan isi .env.
15 func NewPool(ctx context.Context) (*pgxpool.Pool, error) {
16     dsn := url.URL{
17         Scheme: "postgres",
18         User: url.UserPassword(
19             config.GetEnv("DB_USER", "postgres"),
20             config.GetEnv("DB_PASSWORD", ""),
21         ),
22         Host:     config.GetEnv("DB_HOST", "localhost") + ":" + config.GetEnv("DB_PORT", "5432"),
23         Path:     config.GetEnv("DB_NAME", "siakad_mini"),
24         RawQuery: "sslmode=" + config.GetEnv("DB_SSLMODE", "disable"),
25     }
26
27     poolConfig, err := pgxpool.ParseConfig(dsn.String())
28     if err != nil {
29         return nil, fmt.Errorf("konfigurasi database tidak valid: %w", err)
30     }
31     poolConfig.MaxConns = 10
32     poolConfig.MinConns = 2
33     poolConfig.MaxConnIdleTime = 5 * time.Minute
34
35     pool, err := pgxpool.NewWithConfig(ctx, poolConfig)
36     if err != nil {
37         return nil, fmt.Errorf("gagal membuat pool: %w", err)
38     }
39
40     // Pastikan database benar-benar bisa dihubungi.
41     pingCtx, cancel := context.WithTimeout(ctx, 5*time.Second)
42     defer cancel()
43     if err := pool.Ping(pingCtx); err != nil {
44         pool.Close()
45         return nil, fmt.Errorf("database tidak dapat dihubungi: %w", err)
46     }
47
48     return pool, nil
49 }
```

**Gambar 2.19** Isi File database/postgres.go

File helper/security.go berisi fungsi HashPassword dan CheckPassword. Saya memakai `bcryptCost = 12` karena bcrypt memang sengaja dibuat

lambat. Semakin besar cost-nya, semakin mahal usaha penyerang untuk menebak password dengan brute force.

```
helper > security.go
1 package helper
2
3 import "golang.org/x/crypto/bcrypt"
4
5 // bcryptCost menentukan seberapa lambat proses hashing (makin besar makin aman, makin lambat).
6 const bcryptCost = 12
7
8 // HashPassword mengubah password biasa menjadi hash bcrypt.
9 func HashPassword(plain string) (string, error) {
10     hashed, err := bcrypt.GenerateFromPassword([]byte(plain), bcryptCost)
11     if err != nil {
12         return "", err
13     }
14     return string(hashed), nil
15 }
16
17 // CheckPassword mencocokkan password biasa dengan hash yang tersimpan.
18 func CheckPassword(hash, plain string) bool {
19     return bcrypt.CompareHashAndPassword([]byte(hash), []byte(plain)) == nil
20 }
```

**Gambar 2.20** Isi File *helper/security.go*

Seluruh proses seeder saya jalankan di dalam satu transaction. Di awal ada perintah **TRUNCATE ... RESTART IDENTITY CASCADE** yang mengosongkan keempat tabel dan mengembalikan id ke 1. Tujuannya supaya seeder aman dijalankan berkali-kali tanpa menghasilkan data ganda. Setelah itu akun admin dibuat dengan password admin12345 yang langsung di-hash.

```

cmd > seed > main.go
29 func main() {
30     config.LoadEnv()
31     ctx := context.Background()
32
33     pool, err := database.NewPool(ctx)
34     if err != nil {
35         log.Fatal(err)
36     }
37     defer pool.Close()
38
39     tx, err := pool.Begin(ctx)
40     if err != nil {
41         log.Fatal(err)
42     }
43     defer tx.Rollback(ctx) // diabaikan bila sudah Commit
44
45     // Kosongkan semua tabel supaya seeder bisa dijalankan berulang kali.
46     if _, err := tx.Exec(ctx,
47         `TRUNCATE enrollments, students, courses, users RESTART IDENTITY CASCADE`); err != nil {
48         log.Fatal("gagal mengosongkan tabel: ", err)
49     }
50
51     // ----- 1 ADMIN -----
52     adminHash, err := helper.HashPassword("admin12345")
53     if err != nil {
54         log.Fatal(err)
55     }
56     if _, err := tx.Exec(ctx,
57         `INSERT INTO users (email, password, role) VALUES ($1, $2, 'admin')`,
58         "admin@siakad.ac.id", adminHash); err != nil {
59         log.Fatal("gagal insert admin: ", err)
60     }

```

**Gambar 2.21** *Seeder Bagian Awal: Transaction, TRUNCATE, dan Akun Admin*

Data 20 mahasiswa saya buat dengan IPK yang sengaja bervariasi, supaya ketiga batas SKS (24, 21, dan 18) bisa diuji. Untuk setiap mahasiswa, seeder membuat akun di tabel users dengan password awal sama dengan NIM yang di-hash, lalu id hasil RETURNING id dipakai untuk mengisi kolom user\_id di tabel students.

```

cmd > seed > main.go
86     }
87
88     for _, s := range students {
89         hash, err := helper.HashPassword(s.NIM)
90         if err != nil {
91             log.Fatal(err)
92         }
93
94         var userID int
95         email := s.NIM + "@student.siakad.ac.id"
96         if err := tx.QueryRow(ctx,
97             `INSERT INTO users (email, password, role) VALUES ($1, $2, 'mahasiswa') RETURNING id`,
98             email, hash).Scan(&userID); err != nil {
99             log.Fatal("gagal insert user mahasiswa: ", err)
100         }
101
102         if _, err := tx.Exec(ctx,
103             `INSERT INTO students (user_id, nim, nama, prodi, angkatan, ipk_terakhir)
104             VALUES ($1, $2, $3, $4, $5, $6)`,
105             userID, s.NIM, s>Nama, s.Prodi, s.Angkatan, s.IPK); err != nil {
106             log.Fatal("gagal insert student: ", err)
107         }
108     }

```

**Gambar 2.22** *Seeder Bagian Data dan Perulangan 20 Mahasiswa*

Sepuluh mata kuliah saya buat dengan SKS dan kuota yang berbeda-beda. Khusus MK010 (Komputasi Awan), kuotanya sengaja saya isi 2 saja, supaya skenario kuota penuh gampang diuji. Kalau semua insert berhasil, transaction di-commit. Kalau ada satu saja yang gagal, defer tx.Rollback akan membatalkan semuanya.

```

cmd > seed > main.go

109
110 // ----- 10 MATA KULIAH -----
111 // MK010 sengaja kuotanya kecil (2) supaya mudah menguji "kuota penuh".
112 courses := []seedCourse{
113     {"MK001", "Pemrograman Backend Lanjut", 4, 5, 40},
114     {"MK002", "Basis Data Lanjut", 3, 5, 40},
115     {"MK003", "Rekayasa Perangkat Lunak", 3, 5, 35},
116     {"MK004", "Jaringan Komputer", 3, 3, 35},
117     {"MK005", "Kecerdasan Buatan", 3, 5, 30},
118     {"MK006", "Interaksi Manusia dan Komputer", 2, 3, 30},
119     {"MK007", "Keamanan Informasi", 3, 5, 30},
120     {"MK008", "Statistika", 2, 3, 40},
121     {"MK009", "Manajemen Proyek TI", 2, 7, 25},
122     {"MK010", "Komputasi Awan", 3, 7, 2},
123 }
124
125 for _, c := range courses {
126     if _, err := tx.Exec(ctx,
127         `INSERT INTO courses (kode_mk, nama_mk, sks, semester, kuota) VALUES ($1, $2, $3, $4, $5)`,
128         c.Kode, c>Nama, c.SKS, c.Semester, c.Kuota); err != nil {
129         log.Fatal("gagal insert course: ", err)
130     }
131 }
132
133 if err := tx.Commit(ctx); err != nil {
134     log.Fatal("gagal commit: ", err)
135 }
136
137 fmt.Println("Seeder selesai:")
138 fmt.Println("  - 1 admin      : admin@siakad.ac.id / admin12345")
139 fmt.Printf("  - %d mahasiswa : <nim>@student.siakad.ac.id / password = nim\n", len(students))
140 fmt.Printf("  - %d mata kuliah\n", len(courses))
141 }

```

**Gambar 2.23** *Seeder Bagian Mata Kuliah dan Commit Transaction*

Seeder saya jalankan dengan `go run ./cmd/seed`. Prosesnya makan waktu beberapa detik karena ada 21 password yang di-hash dengan bcrypt. Hasilnya saya cek dengan query jumlah user per role, dan didapat 1 admin dan 20 mahasiswa, sesuai ketentuan minimal UTS.

```

PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE  PORTS
• ochadellafitriani@MacBook-Air-ocha siakad-mini % go mod tidy
go run ./cmd/seed
Seeder selesai:
  - 1 admin      : admin@siakad.ac.id / admin12345
  - 20 mahasiswa : <nim>@student.siakad.ac.id / password = nim
  - 10 mata kuliah

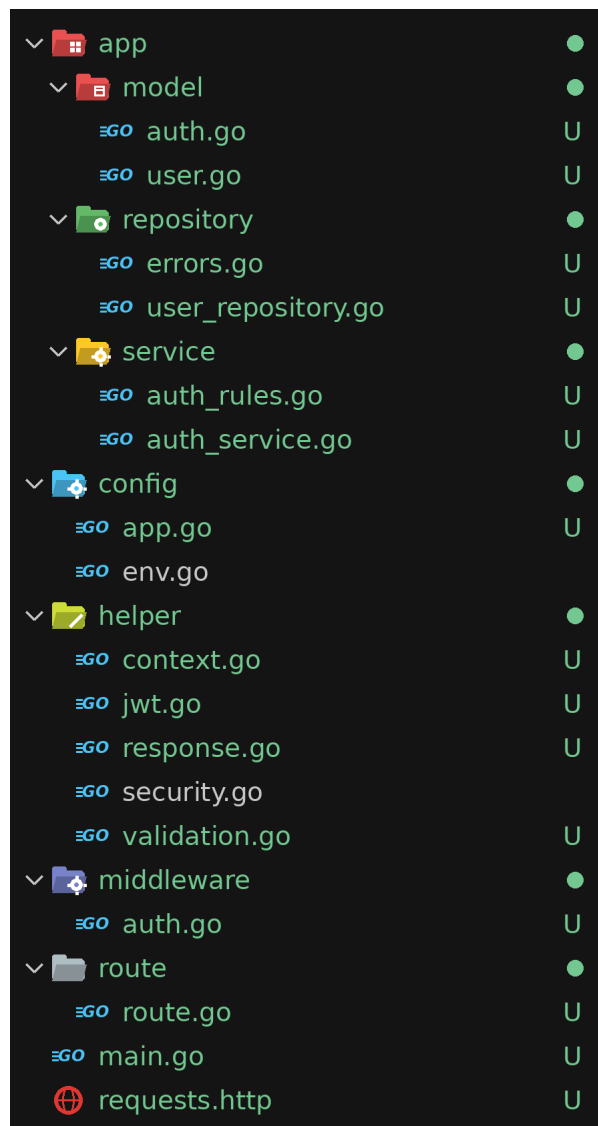
• ochadellafitriani@MacBook-Air-ocha siakad-mini % psql -d siakad_mini -c "SELECT role, COUNT(*) FROM users GROUP BY role;"
role | count
-----+-----
admin |      1
mahasiswa |     20
(2 rows)
○ ochadellafitriani@MacBook-Air-ocha siakad-mini %

```

**Gambar 2.24** *Menjalankan Seeder dan Memverifikasi Jumlah Data*

## 2.7 Langkah 7 Autentikasi: Login dan Profil (Endpoint 1 dan 2)

Di langkah ini saya membuat server Fiber pertama yang bisa dijalankan, sekaligus dua endpoint autentikasi. Autentikasi saya kerjakan paling awal karena semua endpoint lain butuh token. File-file baru di langkah ini bisa dilihat di Gambar 2.25.



**Gambar 2.25** File Baru untuk Server dan Autentikasi

File helper/response.go saya buat untuk menyeragamkan semua response JSON sesuai format dari soal, yaitu field success, message, data, meta (untuk data yang pakai pagination), dan errors (untuk error validasi). Dengan helper ini, setiap

service cukup memanggil OK, Created, NoContent, Fail, atau FailValidation, jadi tidak ada endpoint yang bikin format JSON-nya sendiri-sendiri.

```
helper > response.go
1 package helper
2
3 import "github.com/gofiber/fiber/v2"
4
5 // Response adalah bentuk JSON seragam untuk semua endpoint.
6 type Response struct {
7     Success bool          `json:"success"`
8     Message string         `json:"message"`
9     Data    any             `json:"data,omitempty"`
10    Meta    *Meta           `json:"meta,omitempty"`
11    Errors  map[string][]string `json:"errors,omitempty"`
12 }
13
14 // Meta berisi informasi pagination untuk response berbentuk daftar.
15 type Meta struct {
16     CurrentPage int `json:"current_page"`
17     PerPage     int `json:"per_page"`
18     Total       int `json:"total"`
19     LastPage    int `json:"last_page"`
20 }
21
22 // OK -> 200
23 func OK(c *fiber.Ctx, message string, data any) error {
24     return c.Status(fiber.StatusOK).JSON(Response{Success: true, Message: message, Data: data})
25 }
26
27 // OKList -> 200 + meta pagination
```

**Gambar 2.26** Struktur Response Seragam pada helper/response.go

```
helper > response.go
28 func OKList(c *fiber.Ctx, message string, data any, meta Meta) error {
29     return c.Status(fiber.StatusOK).JSON(Response{Success: true, Message: message, Data: data, Meta: &meta})
30 }
31
32 // Created -> 201
33 func Created(c *fiber.Ctx, message string, data any) error {
34     return c.Status(fiber.StatusCreated).JSON(Response{Success: true, Message: message, Data: data})
35 }
36
37 // NoContent -> 204 (tanpa body)
38 func NoContent(c *fiber.Ctx) error {
39     return c.SendStatus(fiber.StatusNoContent)
40 }
41
42 // Fail -> status error apa pun (401, 403, 404, 409, 429, 500, ...)
43 func Fail(c *fiber.Ctx, status int, message string) error {
44     return c.Status(status).JSON(Response{Success: false, Message: message})
45 }
46
47 // FailValidation -> 422 + daftar error per field
48 func FailValidation(c *fiber.Ctx, errs map[string][]string) error {
49     return c.Status(fiber.StatusUnprocessableEntity).JSON(Response{
50         Success: false,
51         Message: "Validasi gagal",
52         Errors:  errs,
53     })
54 }
```

**Gambar 2.27** Fungsi Pembentuk Response pada helper/response.go

File helper/jwt.go bertugas membuat dan memeriksa access token. Di dalam token saya simpan id, email, dan role pengguna, jadi middleware tidak perlu query ke database untuk tahu role si pemanggil. Di fungsi Parse, algoritma token dicek dulu: token yang tidak memakai **SigningMethodHMAC** langsung ditolak, untuk mencegah serangan yang mengganti algoritma token.

```
helper > jwt.go
40 // GenerateAccess membuat access token untuk user yang berhasil login.
41 func (m *JWTManager) GenerateAccess(u model.User) (string, error) {
42     now := time.Now()
43     claims := accessClaims{
44         UserID: u.ID,
45         Email:   u.Email,
46         Role:    u.Role,
47         RegisteredClaims: jwt.RegisteredClaims{
48             Subject:  fmt.Sprintf(u.ID),
49             Issuer:    m.issuer,
50             IssuedAt:  jwt.NewNumericDate(now),
51             ExpiresAt: jwt.NewNumericDate(now.Add(m.accessTTL)),
52         },
53     }
54     token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
55     return token.SignedString(m.secret)
56 }
57
58 // Parse memeriksa token dan mengembalikan identitas pemiliknya.
59 func (m *JWTManager) Parse(tokenString string) (model.AuthUser, error) {
60     claims := &accessClaims{}
61     _, err := jwt.ParseWithClaims(tokenString, claims,
62         func(t *jwt.Token) (any, error) {
63             // Tolak token yang memakai algoritma selain HMAC.
64             if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
65                 return nil, ErrTokenInvalid
66             }
67             return m.secret, nil
68         },
69         jwt.WithIssuer(m.issuer),
70         jwt.WithExpirationRequired(),
71     )
72     if err != nil {
73         if errors.Is(err, jwt.ErrTokenExpired) {
74             return model.AuthUser{}, ErrTokenExpired
75         }
76         return model.AuthUser{}, ErrTokenInvalid
77     }
78     return model.AuthUser{ID: claims.UserID, Email: claims.Email, Role: claims.Role}, nil
79 }
```

**Gambar 2.28** Pembuatan dan Pemeriksaan Token pada helper/jwt.go

Di repository user ada satu kondisi khusus bernama `activeUserCondition`. Kondisi ini memastikan akun mahasiswa yang sudah di-soft delete **tidak bisa**

login, sesuai ketentuan endpoint 7. Kondisi yang sama saya pakai di FindActiveByEmail (untuk login) dan FindActiveByID (untuk endpoint /me).

```
app > repository > user_repository.go
28 // Kondisi "aktif": user BUKAN mahasiswa yang sudah di-soft delete.
29 const activeUserCondition = `NOT EXISTS (
30     SELECT 1 FROM students s WHERE s.user_id = u.id AND s.deleted_at IS NOT NULL
31 )`
32
33 func (r *userPostgresRepository) FindActiveByEmail(ctx context.Context, email string) (model.User, error) {
34     var u model.User
35     err := r.pool.QueryRow(ctx,
36         `SELECT u.id, u.email, u.password, u.role, u.created_at
37         FROM users u
38         WHERE u.email = $1 AND `+activeUserCondition,
39         email,
40     ).Scan(&u.ID, &u.Email, &u.Password, &u.Role, &u.CreatedAt)
41     if errors.Is(err, pgx.ErrNoRows) {
42         return model.User{}, ErrNotFound
43     }
44     return u, err
45 }
```

**Gambar 2.29** Kondisi Akun Aktif pada *app/repository/user\_repository.go*

Aturan validasi login saya pisahkan di file *app/service/auth\_rules.go* dan sama sekali tidak mengimpor Fiber, mengikuti prinsip pemisahan business rules di Modul 4. Aturannya: email wajib diisi dan formatnya harus benar, password wajib diisi minimal 8 karakter.

```

app > service > auth_rules.go
1  package service
2
3  import (
4      "strings"
5
6      "siakad-mini/app/model"
7      "siakad-mini/helper"
8  )
9
10 // ValidateLogin berisi aturan validasi login. Tidak mengimpor Fiber
11 // sehingga mudah dibuat unit test-nya.
12 func ValidateLogin(req model.LoginRequest) helper.ValidationErrors {
13     errs := helper.ValidationErrors{}
14
15     if strings.TrimSpace(req.Email) == "" {
16         errs.Add("email", "Email wajib diisi")
17     } else if !helper.IsValidEmail(req.Email) {
18         errs.Add("email", "Format email tidak valid")
19     }
20
21     if req.Password == "" {
22         errs.Add("password", "Password wajib diisi")
23     } else if len(req.Password) < 8 {
24         errs.Add("password", "Password minimal 8 karakter")
25     }
26
27     return errs
28 }

```

**Gambar 2.30** Aturan Validasi Login pada *app/service/auth\_rules.go*

Fungsi Login di *app/service/auth\_service.go* berjalan dengan urutan berikut: membaca body JSON, memvalidasi, mencari user yang aktif, mencocokkan password dengan bcrypt, lalu membuat token. Pesan error untuk email yang tidak terdaftar dan password yang salah sengaja saya samakan, yaitu "Email atau password salah", supaya orang luar tidak bisa menebak email mana yang terdaftar. Body-nya saya baca langsung dengan `json.Unmarshal`, dan alasan perubahan ini saya jelaskan di BAB IV.

```

app > service > auth_service.go
24 // Login -> POST /api/v1/auth/login
25 func (s *AuthService) Login(c *fiber.Ctx) error {
26     var req model.LoginRequest
27     // Body dibaca langsung sebagai JSON (tidak bergantung pada header Content-Type).
28     if err := json.Unmarshal(c.Body(), &req); err != nil {
29         return helper.FailValidation(c, map[string][]string{
30             "body": {"Body harus berupa JSON yang valid"},
31         })
32     }
33     req.Email = strings.ToLower(strings.TrimSpace(req.Email))
34
35     if errs := ValidateLogin(req); errs.HasErrors() {
36         return helper.FailValidation(c, errs)
37     }
38
39     user, err := s.users.FindActiveByEmail(c.UserContext(), req.Email)
40     if errors.Is(err, repository.ErrNotFound) {
41         // Pesan sengaja sama dengan "password salah" agar penyerang
42         // tidak bisa menebak email mana yang terdaftar.
43         return helper.Fail(c, fiber.StatusUnauthorized, "Email atau password salah")
44     }
45     if err != nil {
46         return err // diteruskan ke ErrorHandler -> 500
47     }
48
49     if !helper.CheckPassword(user.Password, req.Password) {
50         return helper.Fail(c, fiber.StatusUnauthorized, "Email atau password salah")
51     }
52
53     token, err := s.jwt.GenerateAccess(user)
54     if err != nil {
55         return err
56     }
57
58     return helper.OK(c, "Login berhasil", model.LoginResponse{
59         AccessToken: token,
60         TokenType:   "Bearer",
61         ExpiresIn:   int(s.jwt.AccessTTL().Seconds()),
62         User:        model.UserResponse{ID: user.ID, Email: user.Email, Role: user.Role},
63     })
64 }

```

**Gambar 2.31** Fungsi Login pada `app/service/auth_service.go`

Fungsi Me mengambil identitas dari token, lalu mengecek lagi ke database bahwa akunnya masih aktif. Kalau role-nya mahasiswa, data nim, nama, prodi, dan angkatan ikut ditampilkan di field student, sesuai rincian endpoint 2.

```

app > service > auth_service.go
66 // Me -> GET /api/v1/auth/me
67 func (s *AuthService) Me(c *fiber.Ctx) error {
68     authUser, ok := helper.CurrentUser(c)
69     if !ok {
70         return helper.Fail(c, fiber.StatusUnauthorized, "Token tidak valid")
71     }
72
73     user, err := s.users.FindActiveByID(c.UserContext(), authUser.ID)
74     if errors.Is(err, repository.ErrNotFound) {
75         // Contoh: akun mahasiswa sudah di-soft delete setelah token dibuat.
76         return helper.Fail(c, fiber.StatusUnauthorized, "Akun tidak aktif atau tidak ditemukan")
77     }
78     if err != nil {
79         return err
80     }
81
82     resp := model.MeResponse{ID: user.ID, Email: user.Email, Role: user.Role}
83
84     if user.Role == model.RoleMahasiswa {
85         student, err := s.users.FindStudentBriefByUserID(c.UserContext(), user.ID)
86         if err != nil && !errors.Is(err, repository.ErrNotFound) {
87             return err
88         }
89         resp.Student = student
90     }
91
92     return helper.OK(c, "Profil pengguna berhasil diambil", resp)
93 }

```

**Gambar 2.32** Fungsi Me pada app/service/auth\_service.go

File middleware/auth.go berisi tiga "penjaga". RequireAuth mengecek header Authorization: Bearer <token> dan membalas 401 kalau token tidak ada, salah, atau sudah kedaluwarsa. Lalu ada **RequireRole**, middleware baru di luar materi Modul 5, yang membalas 403 kalau role pemanggil tidak termasuk role yang diizinkan.

```

middleware > auth.go
1 package middleware
2
3 import (
4     "errors"
5     "strings"
6     "time"
7
8     "github.com/gofiber/fiber/v2"
9     "github.com/gofiber/fiber/v2/middleware/limiter"
10
11     "siakad-mini/helper"
12 )
13
14 // RequireAuth memastikan request membawa header "Authorization: Bearer <token>" yang valid.
15 func RequireAuth(jwtManager *helper.JWTManager) fiber.Handler {
16     return func(c *fiber.Ctx) error {
17         header := c.Get(fiber.HeaderAuthorization)
18         if header == "" {
19             return helper.Fail(c, fiber.StatusUnauthorized, "Token tidak ditemukan")
20         }
21
22         parts := strings.SplitN(header, " ", 2)
23         if len(parts) != 2 || !strings.EqualFold(parts[0], "Bearer") || strings.TrimSpace(parts[1]) == "" {
24             return helper.Fail(c, fiber.StatusUnauthorized, "Format token harus: Bearer <token>")
25         }
26
27         authUser, err := jwtManager.Parse(strings.TrimSpace(parts[1]))
28         if errors.Is(err, helper.ErrTokenExpired) {
29             return helper.Fail(c, fiber.StatusUnauthorized, "Token sudah kedaluwarsa")
30         }
31         if err != nil {
32             return helper.Fail(c, fiber.StatusUnauthorized, "Token tidak valid")
33         }
34
35         helper.SetAuthUser(c, authUser)
36         return c.Next()
37     }
38 }
39
40 // RequireRole hanya mengizinkan role tertentu. Dipasang SETELAH RequireAuth.

```

**Gambar 2.33** *Middleware RequireAuth pada middleware/auth.go*

Penjaga ketiga adalah LoginRateLimiter, yang memakai middleware limiter bawaan Fiber dengan batas 5 kali per menit per alamat IP. Opsi **SkipSuccessfulRequests: true** membuat hanya login yang gagal yang dihitung. Jadi percobaan gagal yang keenam dalam satu menit dibalas 429 Too Many Requests, sementara pengguna yang berhasil login tidak ikut terkena batasan.

```

middleware > auth.go
41 func RequireRole(roles ...string) fiber.Handler {
42     return func(c *fiber.Ctx) error {
43         user, ok := helper.CurrentUser(c)
44         if !ok {
45             return helper.Fail(c, fiber.StatusUnauthorized, "Token tidak valid")
46         }
47         for _, r := range roles {
48             if user.Role == r {
49                 return c.Next()
50             }
51         }
52         return helper.Fail(c, fiber.StatusForbidden, "Anda tidak memiliki akses ke resource ini")
53     }
54 }
55
56 // LoginRateLimiter: maksimal 5 percobaan login GAGAL per menit per alamat IP.
57 // Percobaan ke-6 dibalas 429.
58 func LoginRateLimiter() fiber.Handler {
59     return limiter.New(limiter.Config{
60         Max: 5,
61         Expiration: 1 * time.Minute,
62         SkipSuccessfulRequests: true, // login yang berhasil tidak dihitung
63         KeyGenerator: func(c *fiber.Ctx) string {
64             return c.IP()
65         },
66         LimitReached: func(c *fiber.Ctx) error {
67             return helper.Fail(c, fiber.StatusTooManyRequests,
68                 "Terlalu banyak percobaan login gagal. Coba lagi dalam 1 menit")
69         },
70     })
71 }

```

**Gambar 2.34** *Middleware RequireRole dan LoginRateLimiter*

File config/app.go membuat instance Fiber sekaligus ErrorHandler terpusat. Kalau ada error yang tidak ditangani di service, detailnya hanya dicatat di log server, sedangkan klien cukup menerima pesan umum "Terjadi kesalahan pada server" dengan status 500. Middleware **recover** juga saya pasang supaya panic tidak mematikan server dan stack trace tidak bocor ke klien.

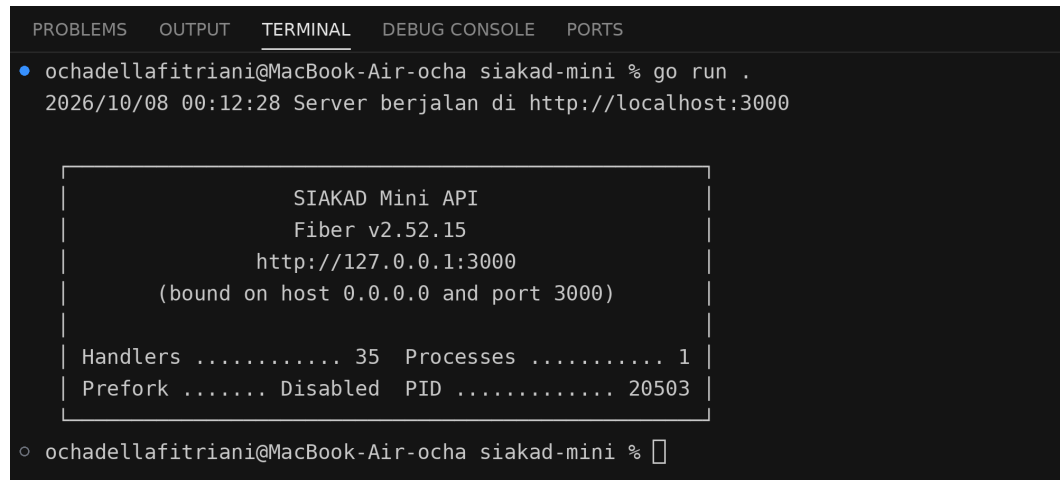
```

config > app.go
1 package config
2
3 import (
4     "errors"
5     "log"
6
7     "github.com/gofiber/fiber/v2"
8     "github.com/gofiber/fiber/v2/middleware/logger"
9     "github.com/gofiber/fiber/v2/middleware/recover"
10    "github.com/gofiber/fiber/v2/middleware/requestid"
11
12    "siakad-mini/helper"
13 )
14
15 // NewApp membuat instance Fiber beserta middleware global dan error handler.
16 func NewApp() *fiber.App {
17     app := fiber.New(fiber.Config{
18         AppName: "SIKAD Mini API",
19         // Semua error yang tidak ditangani di service berakhir di sini.
20         ErrorHandler: func(c *fiber.Ctx, err error) error {
21             var fe *fiber.Error
22             if errors.As(err, &fe) {
23                 return helper.Fail(c, fe.Code, fe.Message)
24             }
25             // Detail error hanya dicatat di log server, TIDAK dikirim ke klien.
26             log.Printf("[ERROR] %s %s: %v", c.Method(), c.Path(), err)
27             return helper.Fail(c, fiber.StatusInternalServerError, "Terjadi kesalahan pada server")
28         },
29     })
30
31     app.Use(recover.New()) // panic -> 500, server tidak mati, stack trace tidak bocor
32     app.Use(requestid.New())
33     app.Use(logger.New(logger.Config{
34         Format: "${time} | ${locals:requestid} | ${status} | ${latency} | ${method} ${path}\n",
35     }))
36
37     return app
38 }

```

**Gambar 2.35** Instance Fiber dan Error Handler pada config/app.go

Setelah route dan main.go selesai dirakit, server saya jalankan dengan `go run .` dan muncul kotak SIAKAD Mini API di port 3000. Sebagai catatan, Gambar 2.36 saya ambil setelah semua endpoint selesai, jadi jumlah handler yang terdaftar sudah 35.



```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
• ochadellafitriani@MacBook-Air-ocha siakad-mini % go run .
2026/10/08 00:12:28 Server berjalan di http://localhost:3000

      SIAKAD Mini API
      Fiber v2.52.15
      http://127.0.0.1:3000
      (bound on host 0.0.0.0 and port 3000)

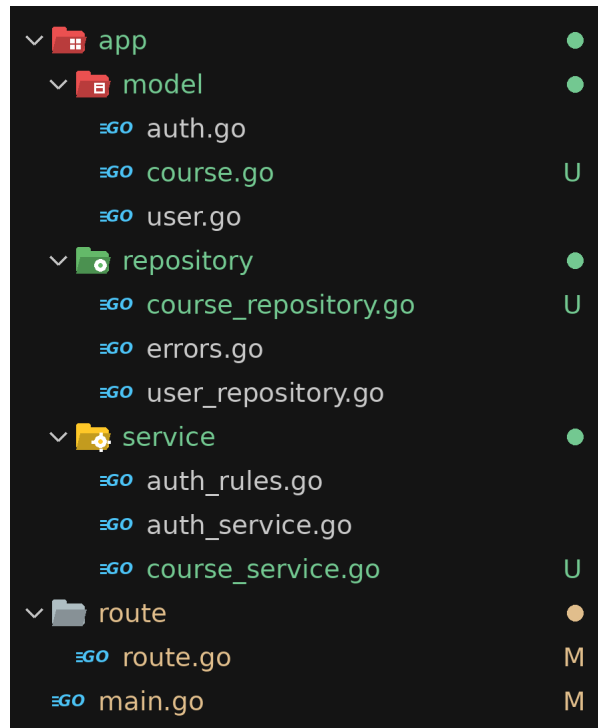
| Handlers ..... 35  Processes ..... 1 |
| Prefork ..... Disabled  PID ..... 20503 |

○ ochadellafitriani@MacBook-Air-ocha siakad-mini %
```

**Gambar 2.36** *Server SIAKAD Mini API Berjalan di Port 3000*

## 2.8 Langkah 8 Daftar Mata Kuliah (Endpoint 8)

Endpoint GET /courses saya kerjakan lebih dulu sebelum CRUD mahasiswa, karena paling sederhana dan bisa diakses semua role. Buat file course.go di folder app/model/, course\_repository.go di folder app/repository/, dan course\_service.go di folder app/service/, lalu ubah route.go dan main.go untuk mendaftarkan service barunya.



**Gambar 2.37** File Baru dan File yang Diubah untuk Endpoint Courses

Kolom terisi tidak saya simpan di tabel courses, tapi **dihitung langsung dari tabel enrollments** pakai LEFT JOIN dan COUNT. Dengan cara ini, angka terisi tidak mungkin berbeda dengan isi KRS yang sebenarnya, dan kalau ada KRS yang dibatalkan, sisa kuota otomatis bertambah tanpa kode tambahan. Klausula WHERE-nya disusun dinamis, tapi nilai filternya tidak pernah ditempel langsung ke string SQL, melainkan selalu lewat placeholder \$1, \$2, dan seterusnya.

```

app > repository > course_repository.go
26 func (r *coursePostgresRepository) FindAll(ctx context.Context, f model.CourseFilter) ([]model.Course, error) {
27     // "terisi" dihitung langsung dari tabel enrollments.
28     query := `
29         SELECT c.id, c.kode_mk, c.nama_mk, c.sks, c.semester, c.kuota,
30             COUNT(e.id) AS terisi
31         FROM courses c
32         LEFT JOIN enrollments e ON e.course_id = c.id`
33
34     // WHERE disusun dinamis. Nilai filter TIDAK pernah ditempel langsung
35     // ke string SQL, selalu lewat placeholder $1, $2, ... (aman dari SQL injection).
36     var conditions []string
37     var args []any
38
39     if f.Semester > 0 {
40         args = append(args, f.Semester)
41         conditions = append(conditions, fmt.Sprintf("c.semester = %d", len(args)))
42     }
43     if f.Search != "" {
44         args = append(args, "%"+f.Search+"%")
45         conditions = append(conditions,
46             fmt.Sprintf("(c.kode_mk ILIKE %d OR c.nama_mk ILIKE %d)", len(args), len(args)))
47     }
48     if len(conditions) > 0 {
49         query += " WHERE " + strings.Join(conditions, " AND ")
50     }
51
52     query += " GROUP BY c.id"
53     if f.Available {
54         query += " HAVING COUNT(e.id) < c.kuota"
55     }

```

**Gambar 2.38** Query Daftar Mata Kuliah dengan Filter Dinamis

Filter available=true saya terjemahkan menjadi **HAVING COUNT(e.id) < c.kuota**, jadi mata kuliah yang kuotanya sudah penuh tidak ikut tampil. Kalau hasilnya kosong, yang dikembalikan adalah array kosong [] dan bukan null, supaya klien tidak perlu menangani dua bentuk data yang berbeda.

```

app > repository > course_repository.go
56     query += " ORDER BY c.semester, c.kode_mk"
57
58     rows, err := r.pool.Query(ctx, query, args...)
59     if err != nil {
60         return nil, err
61     }
62     defer rows.Close()
63
64     courses := make([]model.Course, 0) // supaya hasil kosong menjadi [] bukan null
65     for rows.Next() {
66         var c model.Course
67         if err := rows.Scan(&c.ID, &c.KodeMK, &c>NamaMK, &c.SKS, &c.Semester, &c.Kuota, &c.Terisi); err != nil {
68             return nil, err
69         }
70         c.SisaKuota = c.Kuota - c.Terisi
71         if c.SisaKuota < 0 {
72             c.SisaKuota = 0
73         }
74         courses = append(courses, c)
75     }
76     return courses, rows.Err()
77 }

```

**Gambar 2.39** Perhitungan Sisa Kuota pada course\_repository.go

```

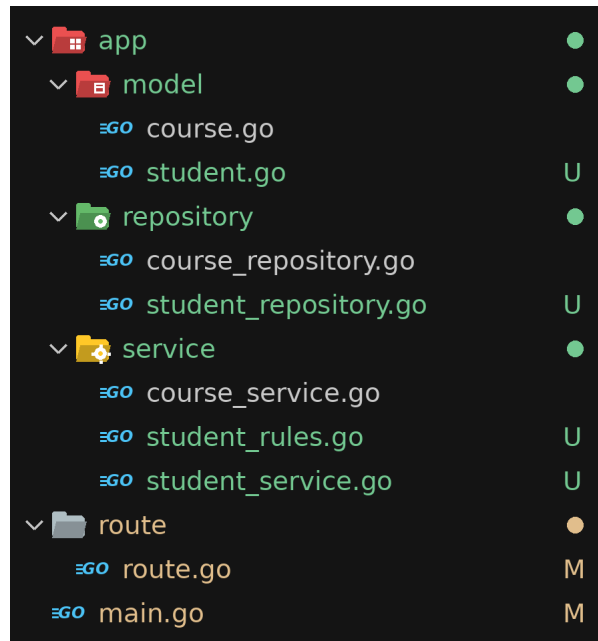
app > service > course_service.go
22 // List -> GET /api/v1/courses?semester=5&search=basis&available=true
23 func (s *CourseService) List(c *fiber.Ctx) error {
24     filter := model.CourseFilter{
25         Search: strings.TrimSpace(c.Query("search")),
26     }
27     errs := helper.ValidationErrors{}
28
29     if v := strings.TrimSpace(c.Query("semester")); v != "" {
30         n, err := strconv.Atoi(v)
31         if err != nil || n < 1 || n > 8 {
32             errs.Add("semester", "Semester harus berupa angka 1 sampai 8")
33         } else {
34             filter.Semester = n
35         }
36     }
37
38     switch strings.ToLower(strings.TrimSpace(c.Query("available"))) {
39     case "":
40         // tidak difilter
41     case "true":
42         filter.Available = true
43     case "false":
44         filter.Available = false
45     default:
46         errs.Add("available", "Nilai available harus true atau false")
47     }
48
49     if errs.HasErrors() {
50         return helper.FailValidation(c, errs)
51     }
52
53     courses, err := s.courses.FindAll(c.UserContext(), filter)
54     if err != nil {
55         return err
56     }
57
58     return helper.OK(c, "Data mata kuliah berhasil diambil", courses)
59 }

```

**Gambar 2.40** Validasi Query Parameter pada *app/service/course\_service.go*

## 2.9 Langkah 9 CRUD Mahasiswa (Endpoint 3 sampai 7)

Langkah ini adalah bagian paling besar dari pengerjaan. Buat file *student.go* di folder *app/model/*, *student\_repository.go* di folder *app/repository/*, serta *student\_rules.go* dan *student\_service.go* di folder *app/service/*.



**Gambar 2.41** File Baru dan File yang Diubah untuk CRUD Mahasiswa

Fungsi **BatasSKS** saya taruh di `student_rules.go` sebagai fungsi murni yang tidak mengimpor Fiber maupun repository. Fungsi ini dipakai di dua tempat: endpoint detail mahasiswa (untuk menampilkan `batas_sks`) dan endpoint KRS (untuk menolak pengambilan yang melebihi batas). Jadi aturan batas SKS cukup ditulis satu kali saja.

```
app > service > student_rules.go
16 // BatasSKS mengembalikan batas SKS per semester berdasarkan IPK terakhir.
17 //
18 //   IPK >= 3.00      -> 24 SKS
19 //   IPK 2.50 - 2.99 -> 21 SKS
20 //   IPK < 2.50      -> 18 SKS
21 func BatasSKS(ipk float64) int {
22     switch {
23     case ipk >= 3.00:
24         return 24
25     case ipk >= 2.50:
26         return 21
27     default:
28         return 18
29     }
30 }
```

**Gambar 2.42** Business Rule Batas SKS pada `student_rules.go`

Validasi untuk POST dan PUT memakai fungsi yang sama, yaitu `validateStudentFields`. Bedanya, POST mewajibkan NIM 12 digit dan email yang valid, sedangkan PUT justru **menolak kalau field nim dikirim**, dengan pesan "NIM tidak boleh diubah". Field request-nya sengaja memakai tipe pointer, supaya bisa dibedakan antara field yang tidak dikirim dan field yang dikirim tapi kosong.

```
app > service > student_rules.go
58
59 // ValidateCreateStudent memeriksa body POST /api/v1/students.
60 func ValidateCreateStudent(req model.CreateStudentRequest, currentYear int) helper.ValidationErrors {
61     errs := helper.ValidationErrors{}
62
63     if req.NIM == nil || strings.TrimSpace(*req.NIM) == "" {
64         errs.Add("nim", "NIM wajib diisi")
65     } else if !nimRegex.MatchString(strings.TrimSpace(*req.NIM)) {
66         errs.Add("nim", "NIM harus terdiri dari 12 digit angka")
67     }
68
69     if req.Email == nil || strings.TrimSpace(*req.Email) == "" {
70         errs.Add("email", "Email wajib diisi")
71     } else if !helper.IsValidEmail(strings.TrimSpace(*req.Email)) {
72         errs.Add("email", "Format email tidak valid")
73     }
74
75     validateStudentFields(errs, req>Nama, req.Prodi, req.Angkatan, req.IPKTerakhir, currentYear)
76     return errs
77 }
78
79 // ValidateUpdateStudent memeriksa body PUT /api/v1/students/{id}.
80 func ValidateUpdateStudent(req model.UpdateStudentRequest, currentYear int) helper.ValidationErrors {
81     errs := helper.ValidationErrors{}
82
83     if req.NIM != nil {
84         errs.Add("nim", "NIM tidak boleh diubah")
85     }
86
87     validateStudentFields(errs, req>Nama, req.Prodi, req.Angkatan, req.IPKTerakhir, currentYear)
88     return errs
89 }
```

**Gambar 2.43** Validasi Tambah dan Ubah Mahasiswa pada `student_rules.go`

Query daftar mahasiswa selalu diawali kondisi **s.deleted\_at IS NULL**, jadi mahasiswa yang sudah di-soft delete tidak akan pernah muncul. Nilai sort dari klien juga tidak langsung ditempel ke SQL, tapi dicocokkan dulu dengan whitelist `studentSortColumns`. Dengan begitu, nilai sort selain nama, -nama, ipk\_terakhir, dan -ipk\_terakhir tidak mungkin menyusup ke query.

```

app > repository > student_repository.go
50 // Whitelist kolom sort: nilai dari klien TIDAK pernah ditempel langsung ke SQL.
51 var studentSortColumns = map[string]string{
52     "nama":      "s.nama ASC",
53     "-nama":     "s.nama DESC",
54     "ipk_terakhir": "s.ipk_terakhir ASC",
55     "-ipk_terakhir": "s.ipk_terakhir DESC",
56 }
57
58 func (r *studentPostgresRepository) List(ctx context.Context, f model.StudentFilter) ([]model.Student, int, error) {
59     conditions := []string{"s.deleted_at IS NULL"} // yang sudah di-soft delete tidak tampil
60     var args []any
61
62     if f.Prodi != "" {
63         args = append(args, f.Prodi)
64         conditions = append(conditions, fmt.Sprintf("s.prodi ILIKE $%d", len(args)))
65     }
66     if f.Angkatan > 0 {
67         args = append(args, f.Angkatan)
68         conditions = append(conditions, fmt.Sprintf("s.angkatan = $%d", len(args)))
69     }
70     if f.Search != "" {
71         args = append(args, "%"+f.Search+"%")
72         conditions = append(conditions,
73             fmt.Sprintf("(s.nim ILIKE $%d OR s.nama ILIKE $%d)", len(args), len(args)))
74     }
75     where := " WHERE " + strings.Join(conditions, " AND ")
76
77     // 1) Hitung total data (untuk meta pagination)
78     var total int
79     if err := r.pool.QueryRow(ctx,
80         `SELECT COUNT(*) FROM students s`+where, args...).Scan(&total); err != nil {
81         return nil, 0, err
82     }

```

**Gambar 2.44** *Whitelist Sort dan Filter Daftar Mahasiswa*

Pagination-nya saya kerjakan dengan dua query: COUNT(\*) untuk menghitung total data yang nanti mengisi objek meta, lalu SELECT dengan LIMIT dan OFFSET untuk mengambil data di halaman yang diminta.

```

app > repository > student_repository.go
83
84     // 2) Ambil data halaman yang diminta
85     orderBy := "s.id ASC"
86     if col, ok := studentSortColumns[f.Sort]; ok {
87         orderBy = col + ", s.id ASC"
88     }
89     args = append(args, f.PerPage, (f.Page-1)*f.PerPage)
90     query := `SELECT ` + studentColumns + `
91         FROM students s JOIN users u ON u.id = s.user_id` + where +
92         ` ORDER BY ` + orderBy +
93         fmt.Sprintf(" LIMIT %d OFFSET %d", len(args)-1, len(args))
94
95     rows, err := r.pool.Query(ctx, query, args...)
96     if err != nil {
97         return nil, 0, err
98     }
99     defer rows.Close()
100
101     students := make([]model.Student, 0)
102     for rows.Next() {
103         s, err := scanStudent(rows)
104         if err != nil {
105             return nil, 0, err
106         }
107         students = append(students, s)
108     }
109     return students, total, rows.Err()
110 }

```

**Gambar 2.45** *Pagination dengan LIMIT dan OFFSET*

Fungsi Create menyimpan data ke dua tabel sekaligus (users dan students) di dalam **satu transaction**, sesuai ketentuan endpoint 4. Kalau insert ke students gagal, akun di users juga ikut dibatalkan, jadi tidak akan ada akun mahasiswa tanpa data mahasiswanya. Error PostgreSQL 23505 (pelanggaran UNIQUE) diterjemahkan menjadi ErrDuplicate oleh fungsi mapDuplicate.

```

app > repository > student_repository.go
137 // Create membuat akun users + data students dalam SATU transaction.
138 // Bila salah satu gagal, keduanya dibatalkan (rollback).
139 func (r *studentPostgresRepository) Create(ctx context.Context, s model.NewStudent) (model.Student, error) {
140     tx, err := r.pool.Begin(ctx)
141     if err != nil {
142         return model.Student{}, err
143     }
144     defer tx.Rollback(ctx) // tidak berpengaruh bila sudah Commit
145
146     var userID int
147     err = tx.QueryRow(ctx,
148         `INSERT INTO users (email, password, role) VALUES ($1, $2, 'mahasiswa') RETURNING id`,
149         s.Email, s.PasswordHash).Scan(&userID)
150     if err != nil {
151         return model.Student{}, mapDuplicate(err)
152     }
153
154     var studentID int
155     err = tx.QueryRow(ctx,
156         `INSERT INTO students (user_id, nim, nama, prodi, angkatan, ipk_terakhir)
157         VALUES ($1, $2, $3, $4, $5, $6) RETURNING id`,
158         userID, s.NIM, s>Nama, s.Prodi, s.Angkatan, s.IPKTerakhir).Scan(&studentID)
159     if err != nil {
160         return model.Student{}, mapDuplicate(err)
161     }
162
163     if err := tx.Commit(ctx); err != nil {
164         return model.Student{}, err
165     }
166     return r.FindByID(ctx, studentID)
167 }

```

**Gambar 2.46** *Tambah Mahasiswa dalam Satu Transaction*

Fungsi `SoftDelete` tidak menjalankan perintah `DELETE`, melainkan `UPDATE` yang mengisi kolom `deleted_at`. Kalau tidak ada baris yang berubah (`RowsAffected` sama dengan 0), artinya datanya memang tidak ada atau sudah dihapus sebelumnya, dan service membalas 404.

```

app > repository > student_repository.go
168
169 func (r *studentPostgresRepository) Update(ctx context.Context, id int, u model.StudentUpdate) (model.Student, error) {
170     tag, err := r.pool.Exec(ctx,
171         `UPDATE students
172         SET nama = $1, prodi = $2, angkatan = $3,
173         ipk_terakhir = COALESCE($4, ipk_terakhir),
174         updated_at = NOW()
175         WHERE id = $5 AND deleted_at IS NULL`,
176         u.Nama, u.Prodi, u.Angkatan, u.IPKTerakhir, id)
177     if err != nil {
178         return model.Student{}, err
179     }
180     if tag.RowsAffected() == 0 {
181         return model.Student{}, ErrNotFound
182     }
183     return r.FindByID(ctx, id)
184 }
185
186 // SoftDelete tidak menghapus baris, hanya mengisi deleted_at.
187 func (r *studentPostgresRepository) SoftDelete(ctx context.Context, id int) error {
188     tag, err := r.pool.Exec(ctx,
189         `UPDATE students SET deleted_at = NOW(), updated_at = NOW()
190         WHERE id = $1 AND deleted_at IS NULL`, id)
191     if err != nil {
192         return err
193     }
194     if tag.RowsAffected() == 0 {
195         return ErrNotFound
196     }
197     return nil
198 }

```

**Gambar 2.47** *Ubah Data dan Soft Delete Mahasiswa*

Endpoint detail bisa diakses admin maupun mahasiswa, tapi mahasiswa hanya boleh melihat datanya sendiri. Pengecekan ini tidak bisa ditaruh di middleware, karena middleware belum tahu data siapa yang diminta. Makanya, service yang membandingkan `student.UserID` dengan `id dari token`, lalu membalas 403 kalau keduanya beda. Response detailnya juga menampilkan daftar mata kuliah yang diambil, `total_sks`, dan `batas_sks`.

```

app > service > student_service.go
162 // Detail -> GET /api/v1/students/{id} (admin, atau mahasiswa untuk data sendiri)
163 func (s *StudentService) Detail(c *fiber.Ctx) error {
164     id, ok := parseStudentID(c)
165     if !ok {
166         return helper.Fail(c, fiber.StatusNotFound, "Mahasiswa tidak ditemukan")
167     }
168
169     ctx := c.UserContext()
170     student, err := s.students.FindByID(ctx, id)
171     if errors.Is(err, repository.ErrNotFound) {
172         return helper.Fail(c, fiber.StatusNotFound, "Mahasiswa tidak ditemukan")
173     }
174     if err != nil {
175         return err
176     }
177
178     // Mahasiswa hanya boleh melihat datanya sendiri.
179     authUser, _ := helper.CurrentUser(c)
180     if authUser.Role == model.RoleMahasiswa && student.UserID != authUser.ID {
181         return helper.Fail(c, fiber.StatusForbidden, "Anda tidak boleh mengakses data mahasiswa lain")
182     }
183
184     courses, err := s.students.FindEnrolledCourses(ctx, student.ID)
185     if err != nil {
186         return err
187     }
188     totalSKS := 0
189     for _, mk := range courses {
190         totalSKS += mk.SKS
191     }
192
193     return helper.OK(c, "Detail mahasiswa berhasil diambil", model.StudentDetail{
194         Student:    student,
195         MataKuliah: courses,
196         TotalSKS:   totalSKS,
197         BatasSKS:   BatasSKS(student.IPKTerakhir),
198     })
199 }

```

**Gambar 2.48** Pemeriksaan Kepemilikan Data pada Detail Mahasiswa

File route.go bisa dibaca seperti peta hak akses. Grup /students mewajibkan RequireAuth, lalu setiap route diberi penjaga role: adminOnly untuk daftar, tambah, ubah, dan hapus, serta adminOrMahasiswa untuk detail. Gambar 2.49 adalah versi akhir route.go, setelah endpoint KRS ditambahkan di langkah berikutnya.

```

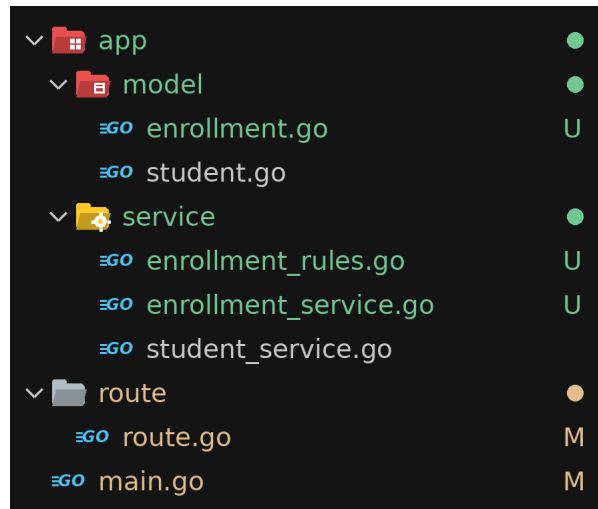
route > route.go
22 func Register(app *fiber.App, deps Deps) {
23     api := app.Group("/api/v1")
24
25     api.Get("/health", func(c *fiber.Ctx) error {
26         return helper.OK(c, "Server berjalan", nil)
27     })
28
29     requireAuth := middleware.RequireAuth(deps.JWT)
30     adminOnly := middleware.RequireRole(model.RoleAdmin)
31     mahasiswaOnly := middleware.RequireRole(model.RoleMahasiswa)
32     adminOrMahasiswa := middleware.RequireRole(model.RoleAdmin, model.RoleMahasiswa)
33
34     // ----- AUTH -----
35     auth := api.Group("/auth")
36     auth.Post("/login", middleware.LoginRateLimiter(), deps.AuthService.Login)
37     auth.Get("/me", requireAuth, deps.AuthService.Me)
38
39     // ----- STUDENTS -----
40     students := api.Group("/students", requireAuth)
41     students.Get("/", adminOnly, deps.StudentService.List)
42     students.Post("/", adminOnly, deps.StudentService.Create)
43     students.Get("/:id", adminOrMahasiswa, deps.StudentService.Detail) // mahasiswa: hanya data sendiri
44     students.Put("/:id", adminOnly, deps.StudentService.Update)
45     students.Delete("/:id", adminOnly, deps.StudentService.Delete)
46
47     // ----- COURSES (semua role) -----
48     api.Get("/courses", requireAuth, deps.CourseService.List)
49
50     // ----- ENROLLMENTS / KRS (mahasiswa) -----
51     enrollments := api.Group("/enrollments", requireAuth, mahasiswaOnly)
52     enrollments.Post("/", deps.EnrollmentService.Create)
53     enrollments.Delete("/:id", deps.EnrollmentService.Delete) // hanya milik sendiri
54
55     // Route yang tidak terdaftar -> 404 dengan format JSON seragam.
56     app.Use(func(c *fiber.Ctx) error {
57         return helper.Fail(c, fiber.StatusNotFound, "Endpoint tidak ditemukan")
58     })
59 }

```

**Gambar 2.49** Pendaftaran Route dan Penjaga Role pada route/route.go

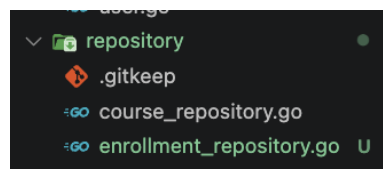
## 2.10 Langkah 10 Kartu Rencana Studi (Endpoint 9 dan 10)

Langkah terakhir adalah endpoint KRS, yang menjalankan semua business rule sekaligus. Buat file enrollment.go di folder app/model/, serta enrollment\_rules.go dan enrollment\_service.go di folder app/service/.



**Gambar 2.50** File Baru untuk Endpoint KRS pada Folder model dan service

Buat file `enrollment_repository.go` di folder `app/repository/`. Di bagian awal file, saya mendefinisikan error khusus untuk setiap pelanggaran business rule. Khusus pelanggaran batas SKS, saya pakai tipe `SKSLimitError` yang membawa angka batas, SKS yang sudah diambil, dan SKS mata kuliahnya, supaya pesan error-nya bisa menyebutkan sisa SKS seperti yang diminta soal.



**Gambar 2.51** File `enrollment_repository.go` pada Folder repository

```

app > repository > -!> enrollment_repository.go > (*enrollmentPostgresRepository).FindByID
1  package repository
2
3  import (
4      "context"
5      "errors"
6      "fmt"
7
8      "github.com/jackc/pgx/v5"
9      "github.com/jackc/pgx/v5/pgxpool"
10
11      "siakad-mini/app/model"
12  )
13
14  var (
15      ErrCourseNotFound = errors.New("mata kuliah tidak ditemukan")
16      ErrAlreadyEnrolled = errors.New("mata kuliah sudah diambil")
17      ErrQuotaFull       = errors.New("kuota mata kuliah penuh")
18      ErrStudentNotActive = errors.New("data mahasiswa tidak aktif")
19  )
20
21  // SKSLimitError dikembalikan bila total SKS melebihi batas.
22  type SKSLimitError struct {
23      Batas      int // batas SKS sesuai IPK
24      Diambil    int // SKS yang sudah diambil di tahun akademik tsb
25      SKSMatkul  int // SKS mata kuliah yang ingin diambil
26  }

```

**Gambar 2.52** Definisi Error dan SKSLimitError pada *enrollment\_repository.go*

Fungsi Enroll menjalankan semua pengecekan di dalam satu transaction. Pertama, baris data mahasiswa dikunci dulu dengan **SELECT ... FOR UPDATE**, supaya dua request dari mahasiswa yang sama tidak bisa sama-sama lolos pengecekan batas SKS.

```

app > repository > enrollment_repository.go
68 // Enroll menjalankan seluruh pengecekan + insert dalam SATU transaction.
69 //
70 // Row locking (SELECT ... FOR UPDATE):
71 // - baris students dikunci -> 2 request dari mahasiswa yang sama tidak bisa
72 //   sama-sama lolos cek batas SKS.
73 // - baris courses dikunci -> 2 mahasiswa yang berebut kursi terakhir harus
74 //   antre; yang kedua baru jalan setelah yang pertama selesai, sehingga
75 //   kuota tidak pernah terlampaui.
76 func (r *enrollmentPostgresRepository) Enroll(ctx context.Context, studentID, courseID int,
77   tahun string, batasSKS func(ipk float64) int) (model.EnrollmentResult, error) {
78
79   var res model.EnrollmentResult
80
81   tx, err := r.pool.Begin(ctx)
82   if err != nil {
83     return res, err
84   }
85   defer tx.Rollback(ctx) // tidak berpengaruh bila sudah Commit
86
87   // 1. Kunci data mahasiswa + ambil IPK
88   var ipk float64
89   err = tx.QueryRow(ctx,
90     `SELECT ipk_terakhir FROM students
91      WHERE id = $1 AND deleted_at IS NULL
92      FOR UPDATE`, studentID).Scan(&ipk)
93   if errors.Is(err, pgx.ErrNoRows) {
94     return res, ErrStudentNotActive
95   }
96   if err != nil {
97     return res, err
98   }

```

**Gambar 2.53** *Awal Transaction dan Penguncian Baris Mahasiswa*

Baris mata kuliahnya juga dikunci dengan FOR UPDATE. Nah, inilah inti dari row locking yang diminta soal. Kalau ada dua mahasiswa yang berebut kursi terakhir di waktu yang sama, request kedua harus menunggu sampai transaction pertama selesai, lalu menghitung ulang kuota dengan data terbaru. Hasilnya, **kuota tidak akan pernah terlampaui**. Setelah itu baru dilakukan pengecekan duplikasi (business rule 2) dan kuota (business rule 3).

```

app > repository > enrollment_repository.go
100 // 2. Kunci mata kuliah + ambil data
101 var kuota int
102 err = tx.QueryRow(ctx,
103     `SELECT kode_mk, nama_mk, sks, kuota FROM courses
104     WHERE id = $1
105     FOR UPDATE`, courseID).Scan(&res.KodeMK, &res>NamaMK, &res.SKS, &kuota)
106 if errors.Is(err, pgx.ErrNoRows) {
107     return res, ErrCourseNotFound
108 }
109 if err != nil {
110     return res, err
111 }
112
113 // 3. Cek duplikasi (business rule 2)
114 var sudahAmbil bool
115 if err := tx.QueryRow(ctx,
116     `SELECT EXISTS (
117         SELECT 1 FROM enrollments
118         WHERE student_id = $1 AND course_id = $2 AND tahun_akademik = $3
119     )`, studentID, courseID, tahun).Scan(&sudahAmbil); err != nil {
120     return res, err
121 }
122 if sudahAmbil {
123     return res, ErrAlreadyEnrolled
124 }
125
126 // 4. Cek kuota (business rule 3)
127 var terisi int
128 if err := tx.QueryRow(ctx,
129     `SELECT COUNT(*) FROM enrollments WHERE course_id = $1`, courseID).Scan(&terisi); err != nil {
130     return res, err
131 }
132 if terisi >= kuota {
133     return res, ErrQuotaFull
134 }

```

**Gambar 2.54** *Penguncian Mata Kuliah, Cek Duplikasi, dan Cek Kuota*

Untuk business rule 1, pengecekan batas SKS dilakukan dengan menjumlahkan SKS yang sudah diambil di tahun akademik yang sama, lalu dibandingkan dengan hasil fungsi BatasSKS. Fungsi BatasSKS ini dioper dari service sebagai parameter, jadi aturan bisnisnya tetap ada di layer service, sedangkan repository cukup mengurus akses data.

```

app > repository > enrollment_repository.go
136 // 5. Cek batas SKS (business rule 1)
137 var diambil int
138 if err := tx.QueryRow(ctx,
139     `SELECT COALESCE(SUM(c.sks), 0)
140     FROM enrollments e JOIN courses c ON c.id = e.course_id
141     WHERE e.student_id = $1 AND e.tahun_akademik = $2`,
142     studentID, tahun).Scan(&diambil); err != nil {
143     return res, err
144 }
145 batas := batasSKS(ipk)
146 if diambil+res.SKS > batas {
147     return res, &SKSLimitError{Batas: batas, Diambil: diambil, SKSMatkul: res.SKS}
148 }
149
150 // 6. Simpan
151 err = tx.QueryRow(ctx,
152     `INSERT INTO enrollments (student_id, course_id, tahun_akademik)
153     VALUES ($1, $2, $3)
154     RETURNING id, created_at`,
155     studentID, courseID, tahun).Scan(&res.ID, &res.CreatedAt)
156 if err != nil {
157     if errors.Is(mapDuplicate(err), ErrDuplicate) {
158         return res, ErrAlreadyEnrolled
159     }
160     return res, err
161 }
162
163 if err := tx.Commit(ctx); err != nil {
164     return res, err
165 }
166
167 res.StudentID = studentID
168 res.CourseID = courseID
169 res.TahunAkademik = tahun
170 res.TotalSKS = diambil + res.SKS
171 res.BatasSKS = batas
172 res.SisaSKS = batas - res.TotalSKS
173 return res, nil
174 }

```

**Gambar 2.55** *Cek Batas SKS, Insert, dan Commit*

Format tahun akademik saya validasi dengan regular expression, ditambah satu pengecekan logis: tahun kedua harus sama dengan tahun pertama ditambah satu. Jadi 2026/2027-Ganjil diterima, sedangkan 2026/2028-Ganjil ditolak.

```

app > service > enrollment_rules.go
12 // Format tahun akademik: 2026/2027-Ganjil atau 2026/2027-Genap
13 var tahunAkademikRegex = regexp.MustCompile(`^([0-9]{4})/([0-9]{4})-(Ganjil|Genap)$`)
14
15 // IsValidTahunAkademik memeriksa format dan memastikan tahun kedua = tahun pertama + 1.
16 func IsValidTahunAkademik(s string) bool {
17     m := tahunAkademikRegex.FindStringSubmatch(s)
18     if m == nil {
19         return false
20     }
21     awal, _ := strconv.Atoi(m[1])
22     akhir, _ := strconv.Atoi(m[2])
23     return akhir == awal+1
24 }
25
26 // ValidateCreateEnrollment memeriksa body POST /api/v1/enrollments.
27 func ValidateCreateEnrollment(req model.CreateEnrollmentRequest) helper.ValidationErrors {
28     errs := helper.ValidationErrors{}
29
30     if req.CourseID == nil {
31         errs.Add("course_id", "course_id wajib diisi")
32     } else if *req.CourseID <= 0 {
33         errs.Add("course_id", "course_id harus berupa angka positif")
34     }
35
36     if req.TahunAkademik == nil || strings.TrimSpace(*req.TahunAkademik) == "" {
37         errs.Add("tahun_akademik", "Tahun akademik wajib diisi")
38     } else if !IsValidTahunAkademik(strings.TrimSpace(*req.TahunAkademik)) {
39         errs.Add("tahun_akademik", "Format tahun akademik harus seperti 2026/2027-Ganjil atau 2026/2027-Genap")
40     }
41
42     return errs
43 }

```

**Gambar 2.56** Validasi Tahun Akademik pada `enrollment_rules.go`

Service bertugas menerjemahkan setiap error dari repository menjadi status code sesuai soal: 409 untuk mata kuliah yang sudah diambil, 422 untuk kuota penuh dan SKS yang melebihi batas (lengkap dengan sisa SKS-nya), serta 422 untuk mata kuliah yang tidak ditemukan.

```

app + service > enrollment_service.go
54     var sksErr *repository.SKSLimitError
55     switch {
56     case err == nil:
57         return helper.Created(c, "Mata kuliah berhasil ditambahkan ke KRS", result)
58
59     case errors.Is(err, repository.ErrCourseNotFound):
60         return helper.FailValidation(c, map[string][]string{
61             "course_id": {"Mata kuliah tidak ditemukan"},
62         })
63
64     case errors.Is(err, repository.ErrAlreadyEnrolled):
65         return helper.Fail(c, fiber.StatusConflict,
66             "Mata kuliah ini sudah diambil pada tahun akademik "+tahun)
67
68     case errors.Is(err, repository.ErrQuotaFull):
69         return helper.FailValidation(c, map[string][]string{
70             "course_id": {"Kuota mata kuliah sudah penuh"},
71         })
72
73     case errors.As(err, &sksErr):
74         return helper.FailValidation(c, map[string][]string{
75             "course_id": {fmt.Sprintf(
76                 "Total SKS melebihi batas. Batas SKS Anda %d, sudah diambil %d, sisa SKS %d, sedangkan mata kuliah ini %d SKS",
77                 sksErr.Batas, sksErr.Diambil, sksErr.Sisa(), sksErr.SKSMatkul)},
78         })
79
80     case errors.Is(err, repository.ErrStudentNotActive):
81         return helper.Fail(c, fiber.StatusForbidden, "Akun mahasiswa tidak aktif")
82
83     default:
84         return err
85     }
86 }

```

**Gambar 2.57** *Pemetaan Error ke Status Code pada enrollment\_service.go*

Sebelum menghapus, pembatalan KRS mengecek dulu kepemilikannya, sesuai business rule 4. Kalau enrollment itu milik mahasiswa lain, service membalas 403. Setelah barisnya dihapus, sisa kuota mata kuliah otomatis bertambah, karena kolom terisi di endpoint courses selalu dihitung dari tabel enrollments.

```

app > service > enrollment_service.go
88 // Delete -> DELETE /api/v1/enrollments/{id} (mahasiswa, hanya milik sendiri)
89 func (s *EnrollmentService) Delete(c *fiber.Ctx) error {
90     id, err := strconv.Atoi(c.Params("id"))
91     if err != nil || id <= 0 {
92         return helper.Fail(c, fiber.StatusNotFound, "Data KRS tidak ditemukan")
93     }
94
95     studentID, err := s.currentStudentID(c)
96     if errors.Is(err, repository.ErrStudentNotActive) {
97         return helper.Fail(c, fiber.StatusForbidden, "Akun mahasiswa tidak aktif")
98     }
99     if err != nil {
100         return err
101     }
102
103     enrollment, err := s.enrollments.FindByID(c.UserContext(), id)
104     if errors.Is(err, repository.ErrNotFound) {
105         return helper.Fail(c, fiber.StatusNotFound, "Data KRS tidak ditemukan")
106     }
107     if err != nil {
108         return err
109     }
110
111     // Business rule 4: hanya boleh mengubah KRS milik sendiri.
112     if enrollment.StudentID != studentID {
113         return helper.Fail(c, fiber.StatusForbidden, "Anda tidak boleh membatalkan KRS milik mahasiswa lain")
114     }
115
116     // Kuota otomatis bertambah, karena "terisi" dihitung dari jumlah baris enrollments.
117     err = s.enrollments.Delete(c.UserContext(), id)
118     if errors.Is(err, repository.ErrNotFound) {
119         return helper.Fail(c, fiber.StatusNotFound, "Data KRS tidak ditemukan")
120     }
121     if err != nil {
122         return err
123     }
124
125     return helper.NoContent(c)
126 }

```

**Gambar 2.58** Pembatalan KRS dengan Pemeriksaan Kepemilikan

Terakhir, main.go merakit semua repository dan service, lalu mengopernya ke route. Urutannya dari bawah ke atas: koneksi database, JWT manager, repository, service, lalu aplikasi Fiber.

```

main.go
36
37 // 3. Repository -> Service
38 userRepo := repository.NewUserRepository(pool)
39 courseRepo := repository.NewCourseRepository(pool)
40 studentRepo := repository.NewStudentRepository(pool)
41 enrollmentRepo := repository.NewEnrollmentRepository(pool)
42
43 authService := service.NewAuthService(userRepo, jwtManager)
44 courseService := service.NewCourseService(courseRepo)
45 studentService := service.NewStudentService(studentRepo)
46 enrollmentService := service.NewEnrollmentService(enrollmentRepo)
47
48 // 4. Aplikasi + route
49 app := config.NewApp()
50 route.Register(app, route.Deps{
51     JWT:          jwtManager,
52     AuthService:   authService,
53     CourseService: courseService,
54     StudentService: studentService,
55     EnrollmentService: enrollmentService,
56 })
57
58 port := config.GetEnv("APP_PORT", "3000")
59 log.Printf("Server berjalan di http://localhost:%s", port)
60 log.Fatal(app.Listen(":" + port))
61 }

```

**Gambar 2.59** Perakitan Repository, Service, dan Route pada main.go

## 2.11 Langkah 11 Dokumentasi dan Riwayat Commit

Sebagai penutup, saya menambahkan file README.md yang berisi cara menjalankan proyek, daftar akun hasil seeder, daftar endpoint, business rules, dan cara pengujian. File requests.http yang berisi 36 request pengujian juga ikut saya commit, dengan catatan token-nya dikembalikan dulu ke tulisan contoh supaya token asli tidak ikut ter-upload. Riwayat commit akhirnya bisa dilihat di Gambar 2.60 dan Tabel 2.1.

```

PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE  PORTS
• ochadellafitriani@MacBook-Air-ocha siakad-mini % git log --oneline
034544c (HEAD -> main, origin/main) fix: isi enrollment repository yang sebelumnya kosong
f0b4223 feat: endpoint KRS ambil dan batal mata kuliah dengan transaction dan row locking
4c86597 feat: CRUD students dengan pagination, filter, dan soft delete
64ba977 feat: endpoint GET courses dengan filter dan sisa kuota
eb9239d feat: endpoint login dan auth/me dengan JWT dan rate limiter
5698598 feat: seeder 1 admin, 20 mahasiswa, 10 mata kuliah
d693f94 feat: migration tabel users, students, courses, enrollments
069ba74 chore: install dependencies dan konfigurasi env
ffaf6cd init: struktur folder proyek SIAKAD Mini
• ochadellafitriani@MacBook-Air-ocha siakad-mini % git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
○ ochadellafitriani@MacBook-Air-ocha siakad-mini % 

```

**Gambar 2.60** Riwayat Commit dan Status Akhir Repositori

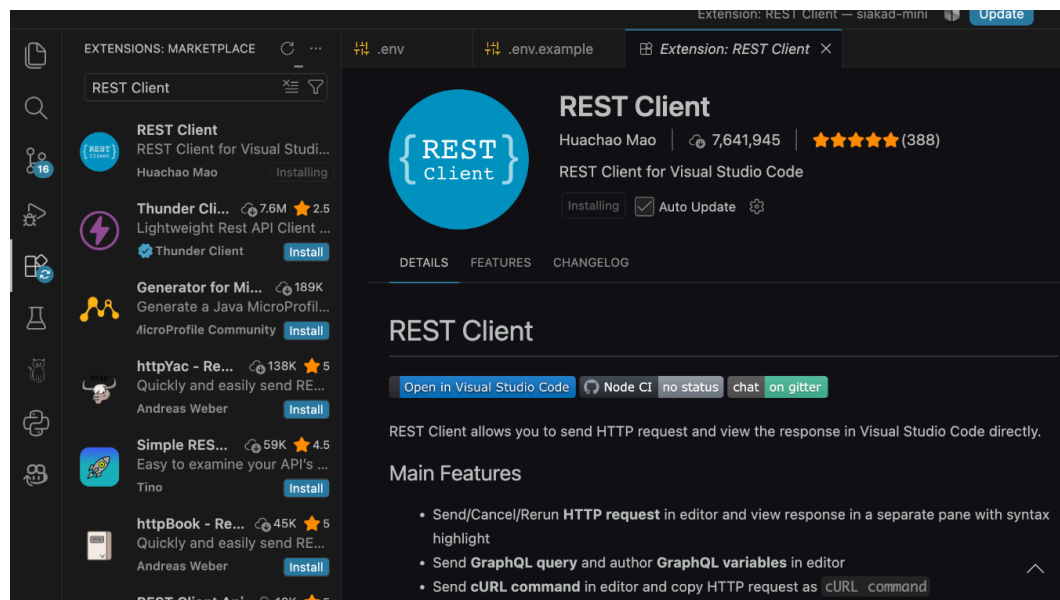
**Tabel 2.1** Riwayat Commit Bertahap di GitHub

| Commit  | Pesan Commit                                                                      | Isi Perubahan                       |
|---------|-----------------------------------------------------------------------------------|-------------------------------------|
| ffaf6cd | init: struktur folder proyek SIAKAD Mini                                          | Struktur folder, .gitignore, go.mod |
| 069ba74 | chore: install dependencies dan konfigurasi env                                   | go.sum, .env.example                |
| d693f94 | feat: migration tabel users, students, courses, enrollments                       | Empat file migration                |
| 5698598 | feat: seeder 1 admin, 20 mahasiswa, 10 mata kuliah                                | Seeder, env, koneksi DB, hashing    |
| eb9239d | feat: endpoint login dan auth/me dengan JWT dan rate limiter                      | Endpoint 1-2, middleware, server    |
| 64ba977 | feat: endpoint GET courses dengan filter dan sisa kuota                           | Endpoint 8                          |
| 4c86597 | feat: CRUD students dengan pagination, filter, dan soft delete                    | Endpoint 3-7                        |
| f0b4223 | feat: endpoint KRS ambil dan batal mata kuliah dengan transaction dan row locking | Endpoint 9-10                       |
| 034544c | fix: isi enrollment repository yang sebelumnya kosong                             | Perbaikan file kosong, README       |

## BAB III PENGUJIAN

### 3.1 Metode Pengujian

Pengujian saya lakukan ke server yang berjalan di `http://localhost:3000` menggunakan extension **REST Client** di VS Code. Alasan saya memilih extension ini: semua request pengujian bisa ditulis dalam satu file teks (`requests.http`) yang ikut tersimpan di repositori, jadi pengujiannya bisa diulang oleh siapa pun yang membuka proyek ini dengan hasil yang konsisten.



**Gambar 3.1** Pemasangan Extension REST Client di VS Code

File `requests.http` berisi 36 request yang saya kelompokkan per endpoint. Di judul setiap request saya tuliskan status code yang diharapkan, misalnya (`-> 403`), jadi untuk mengecek hasilnya cukup dicocokkan dengan baris pertama response.

```

75  ### ===== STUDENTS (pakai tokenAdmin) =====
76
77  ### 14. Daftar mahasiswa (-> 200)
78  Send Request
79  GET {{baseUrl}}/students
80  Authorization: Bearer {{tokenAdmin}}
81
82  ### 15. Pagination + sort IPK tertinggi (-> 200)
83  Send Request
84  GET {{baseUrl}}/students?page=2&per_page=5&sort=-ipk_terakhir
85  Authorization: Bearer {{tokenAdmin}}
86
87  ### 16. Filter prodi + angkatan + search (-> 200)
88  Send Request
89  GET {{baseUrl}}/students?prodi=Sistem Informasi&angkatan=2022&search=rina
90  Authorization: Bearer {{tokenAdmin}}
91
92  ### 17. Daftar mahasiswa sebagai mahasiswa (-> 403)
93  Send Request
94  GET {{baseUrl}}/students
95  Authorization: Bearer {{tokenMhs}}
96
97  ### 18. Tambah mahasiswa (-> 201)
98  Send Request
99  POST {{baseUrl}}/students
100 Authorization: Bearer {{tokenAdmin}}
101 Content-Type: application/json
102
103 {"nim":"187221000099","nama":"Zahra Amelia","email":"zahra@student.siakad.ac.id","prodi":"Sistem Informasi","angkatan":2024,"ipk_terakhir":3.2}
104
105 ### 19. Tambah mahasiswa dengan NIM/email yang sama (-> 422)
106 Send Request
107 POST {{baseUrl}}/students
108 Authorization: Bearer {{tokenAdmin}}
109 Content-Type: application/json
110
111 {"nim":"187221000099","nama":"Zahra Lagi","email":"zahra@student.siakad.ac.id","prodi":"Sistem Informasi","angkatan":2024}
112
113 ### 20. Tambah mahasiswa data tidak valid (-> 422)
114 Send Request
115 POST {{baseUrl}}/students
116 Authorization: Bearer {{tokenAdmin}}
117 Content-Type: application/json
118
119 {"nim":"123","email":"bukan-email","angkatan":2099,"ipk_terakhir":5}
120
121 ### 21. Detail mahasiswa id 1 sebagai admin (-> 200)
122 Send Request
123 GET {{baseUrl}}/students/1
124 Authorization: Bearer {{tokenAdmin}}
125
126 ### 22. Mahasiswa lihat data sendiri, Rina = id 1 (-> 200)
127 Send Request
128 GET {{baseUrl}}/students/1
129 Authorization: Bearer {{tokenMhs}}
130

```

**Gambar 3.2** Isi File *requests.http* Bagian Endpoint Students

Token hasil login saya tempel ke dua variabel di bagian atas file, yaitu @tokenMhs untuk mahasiswa dan @tokenAdmin untuk admin, lalu dipakai ulang oleh request lain lewat {{tokenMhs}} dan {{tokenAdmin}}. Sebelum commit, kedua variabel ini saya kembalikan ke tulisan contoh supaya token aslinya tidak ikut ter-upload. Akun yang saya pakai selama pengujian ada di Tabel 3.1.

```

8  # PLACING: sebelum commit, kembalikan ke tulisan contoh di bawah.
9  # =====
10 @tokenMhs = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1aWQiOiJlImVtYWlsIjoiaMTg3MjI1IiwiaWF0Ij0i
11 @tokenAdmin = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1aWQiOiJlImVtYWlsIjoiaWYWRtaW5Ac
12

```

**Gambar 3.3** Variabel Token Mahasiswa dan Admin pada *requests.http*

**Tabel 3.1** Akun yang Digunakan dalam Pengujian

| Peran     | Email                             | Password     | Keterangan                                |
|-----------|-----------------------------------|--------------|-------------------------------------------|
| Admin     | admin@siakad.ac.id                | admin12345   | Akun admin hasil seeder                   |
| Mahasiswa | 187221000001@student.siakad.ac.id | 187221000001 | Rina Putri, id 1, IPK 3,45 (batas 24 SKS) |

### 3.2 Pengujian Autentikasi

Login mahasiswa dengan email dan password yang benar menghasilkan 200 OK, lengkap dengan access\_token, token\_type Bearer, dan expires\_in 3600 detik (60 menit). Header X-Ratelimit-Remaining tetap 5, karena login yang berhasil tidak dihitung oleh rate limiter.

```

requests.http x README.md auth.go enrollment.go enrollment_repository.g... Response(940ms) x
requests.http x POST /api/v1/auth/login
@baseUrl = http://localhost:3000/api/v1
1 # =====
2 # TOKEN: jalankan request login (nomor 1 / 3), copy isi
3 # "access_token" dari hasilnya, lalu tempel setelah tanda "=",
4 # Token berlaku 60 menit. Kalau "Token sudah kedaluwarsa",
5 # login ulang dan ganti tokenya.
6 # PENTING: sebelum commit, kembalikan ke tulisan contoh di bawah.
7 # =====
8 @tokenMhs = TEMPEL_TOKEN_MAHASISWA_DI_SINI
9 @tokenAdmin = TEMPEL_TOKEN_ADMIN_DI_SINI
10
11 ## 1. Login mahasiswa (BERHASIL -> 200)
12 POST {{baseUrl}}/auth/login
13 Content-Type: application/json
14 {
15   "email": "187221000001@student.siakad.ac.id", "password": "187221000001"
16 }
17
18 ## 2. Profil mahasiswa yang login (-> 200)
19 GET {{baseUrl}}/auth/me
20 Authorization: Bearer {{tokenMhs}}
21
22 ## 3. Login admin (BERHASIL -> 200)
23 POST {{baseUrl}}/auth/login
24 Content-Type: application/json
25 {
26   "email": "admin@siakad.ac.id", "password": "admin12345"
27 }
28
29 ## 4. Profil admin (-> 200)
30 GET {{baseUrl}}/auth/me
31 Authorization: Bearer {{tokenAdmin}}
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
253
```



```
● Response(41ms)
1 HTTP/1.1 200 OK
2 Date: Wed, 07 Oct 2026 17:26:19 GMT
3 Content-Type: application/json
4 Content-Length: 121
5 X-Request-Id: 32f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Profil pengguna berhasil diambil",
11  "data": {
12    "id": 1,
13    "email": "admin@siakad.ac.id",
14    "role": "admin"
15  }
16 }
```

**Gambar 3.7** Pengujian 4: Profil Admin (200)

Password yang salah dibalas 401 dengan pesan umum "Email atau password salah". Di sini nilai X-Ratelimit-Remaining turun jadi 4, yang membuktikan bahwa percobaan gagal ikut dihitung oleh rate limiter.

```
● Response(40ms)
1 HTTP/1.1 401 Unauthorized
2 Date: Wed, 07 Oct 2026 17:27:09 GMT
3 Content-Type: application/json
4 Content-Length: 55
5 X-Request-Id: 33f531c8-8e47-48c9-93d6-4cfb24e64d64
6 X-Ratelimit-Limit: 5
7 X-Ratelimit-Remaining: 4
8 X-Ratelimit-Reset: 60
9 Connection: close
10
11 {
12   "success": false,
13   "message": "Email atau password salah"
14 }
```

**Gambar 3.8** Pengujian 5: Login dengan Password Salah (401)

Kalau email-nya bukan format email dan password-nya kurang dari 8 karakter, server membalas 422 dengan pesan error per field di dalam objek errors.

```
1 POST {{baseUrl}}/api/v1/auth/login
2 @baseUrl = http://localhost:3000/api/v1
3 ## 1. Login mahasiswa (BERHASIL -> 200)
4 # @name loginMhs
5 Send Request
6 POST {{baseUrl}}/auth/login
7 Content-Type: application/json
8 {"email":"187221000001@student.siakad.ac.id","password":"187221000001"}
9
10 ## 2. Profil mahasiswa yang login (-> 200)
11 Send Request
12 GET {{baseUrl}}/auth/me
13 Authorization: Bearer {{loginMhs.response.body.data.access_token}}
14
15 ## 3. Login admin (BERHASIL -> 200)
16 # @name loginAdmin
17 Send Request
18 POST {{baseUrl}}/auth/login
19 Content-Type: application/json
20 {"email":"admin@siakad.ac.id","password":"admin12345"}
21
22 ## 4. Profil admin (-> 200)
23 Send Request
24 GET {{baseUrl}}/auth/me
25 Authorization: Bearer {{loginAdmin.response.body.data.access_token}}
26
27 ## 5. Password salah (-> 401)
28 Send Request
29 POST {{baseUrl}}/auth/login
30 Content-Type: application/json
31 {"email":"admin@siakad.ac.id","password":"salahsalah"}
32
33 ## 6. Validasi gagal (-> 422)
34 Send Request
35 POST {{baseUrl}}/auth/login
36 Content-Type: application/json
37 {"email":"bukan-email","password":"123"}
38
39 ## 7. /me tanpa token (-> 401)
40 Send Request
41 GET {{baseUrl}}/auth/me
```

```
1 HTTP/1.1 422 Unprocessable Entity
2 Date: Wed, 07 Oct 2026 16:26:15 GMT
3 Content-Type: application/json
4 Content-Length: 135
5 X-Request-Id: f89462ee-8839-49fc-b121-a3220e53c5f9
6 X-Ratelimit-Limit: 5
7 X-Ratelimit-Remaining: 4
8 X-Ratelimit-Reset: 60
9 Connection: close
10
11 {
12   "success": false,
13   "message": "Validasi gagal",
14   "errors": {
15     "email": [
16       "Format email tidak valid"
17     ],
18     "password": [
19       "Password minimal 8 karakter"
20     ]
21   }
22 }
```

**Gambar 3.9** Pengujian 6: Validasi Login Gagal (422)

Request tanpa header Authorization dan request dengan token palsu sama-sama dibalas 401, tapi pesannya berbeda, jadi klien bisa tahu penyebabnya.

```
1 HTTP/1.1 401 Unauthorized
2 Date: Wed, 07 Oct 2026 17:28:15 GMT
3 Content-Type: application/json
4 Content-Length: 51
5 X-Request-Id: 34f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Token tidak ditemukan"
11 }
```

**Gambar 3.10** Pengujian 7: Profil tanpa Token (401)

```
● Response(18ms)
1 HTTP/1.1 401 Unauthorized
2 Date: Wed, 07 Oct 2026 17:28:43 GMT
3 Content-Type: application/json
4 Content-Length: 47
5 X-Request-Id: 35f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Token tidak valid"
11 }
```

**Gambar 3.11** *Pengujian 8: Profil dengan Token Palsu (401)*

### 3.3 Pengujian Daftar Mata Kuliah

Endpoint courses menampilkan sepuluh mata kuliah, dan setiap mata kuliah dilengkapi terisi serta sisa\_kuota yang dihitung dari tabel enrollments.

```
● Response(39ms)
1 HTTP/1.1 200 OK
2 Date: Wed, 07 Oct 2026 17:29:12 GMT
3 Content-Type: application/json
4 Content-Length: 1252
5 X-Request-Id: 36f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Data mata kuliah berhasil diambil",
11  "data": [
12    {
13      "id": 4,
14      "kode_mk": "MK004",
15      "nama_mk": "Jaringan Komputer",
16      "sks": 3,
17      "semester": 3,
18      "kuota": 35,
19      "terisi": 0,
20      "siswa_kuota": 35
21    },
22    {
23      "id": 6,
24      "kode_mk": "MK006",
25      "nama_mk": "Interaksi Manusia dan Komputer",
26      "sks": 2,
27      "semester": 3,
28      "kuota": 30,
29      "terisi": 0,
30      "siswa_kuota": 30
31    },
32    {
33      "id": 8,
34      "kode_mk": "MK008",
35      "nama_mk": "Statistika",
36      "sks": 2,
37      "semester": 3,
38      "kuota": 40,
39      "terisi": 0,
40      "siswa_kuota": 40
41    },
42    {
43      "id": 1,
44      "kode_mk": "MK001",
```

**Gambar 3.12** Pengujian 9: Daftar Seluruh Mata Kuliah (200)

Dengan kombinasi filter semester=5 dan search=basis, hasilnya hanya satu mata kuliah, yaitu Basis Data Lanjut.

```
● Response(22ms)
1 HTTP/1.1 200 OK
2 Date: Wed, 07 Oct 2026 17:30:11 GMT
3 Content-Type: application/json
4 Content-Length: 187
5 X-Request-Id: 37f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Data mata kuliah berhasil diambil",
11  "data": [
12    {
13      "id": 2,
14      "kode_mk": "MK002",
15      "nama_mk": "Basis Data Lanjut",
16      "sks": 3,
17      "semester": 5,
18      "kuota": 40,
19      "terisi": 0,
20      "siswa_kuota": 40
21    }
22  ]
23 }
```

**Gambar 3.13** *Pengujian 10: Filter Semester dan Pencarian (200)*

Filter `available=true` hanya menampilkan mata kuliah yang kuotanya belum penuh. Karena saat pengujian ini belum ada mata kuliah yang penuh, hasilnya sama dengan Pengujian 9.

```
● Response(13ms)
1 HTTP/1.1 200 OK
2 Date: Wed, 07 Oct 2026 17:30:42 GMT
3 Content-Type: application/json
4 Content-Length: 1252
5 X-Request-Id: 38f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Data mata kuliah berhasil diambil",
11  "data": [
12    {
13      "id": 4,
14      "kode_mk": "MK004",
15      "nama_mk": "Jaringan Komputer",
16      "sks": 3,
17      "semester": 3,
18      "kuota": 35,
19      "terisi": 0,
20      "siswa_kuota": 35
21    },
22    {
23      "id": 6,
24      "kode_mk": "MK006",
25      "nama_mk": "Interaksi Manusia dan Komputer",
26      "sks": 2,
27      "semester": 3,
28      "kuota": 30,
29      "terisi": 0,
30      "siswa_kuota": 30
31    },
32    {
33      "id": 8,
34      "kode_mk": "MK008",
35      "nama_mk": "Statistika",
36      "sks": 2,
37      "semester": 3,
38      "kuota": 40,
39      "terisi": 0,
40      "siswa_kuota": 40
41    },
42    {
43      "id": 1,
44      "kode_mk": "MK001",
```

**Gambar 3.14** *Pengujian II: Mata Kuliah yang Kuotanya Belum Penuh (200)*

```
● Response(11ms)
1 HTTP/1.1 422 Unprocessable Entity
2 Date: Wed, 07 Oct 2026 17:31:30 GMT
3 Content-Type: application/json
4 Content-Length: 163
5 X-Request-Id: 39f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Validasi gagal",
11  "errors": {
12    "available": [
13      "Nilai available harus true atau false"
14    ],
15    "semester": [
16      "Semester harus berupa angka 1 sampai 8"
17    ]
18  }
19 }
```

**Gambar 3.15** *Pengujian 12: Query Parameter Tidak Valid (422)*

```
● Response(23ms)
1 HTTP/1.1 401 Unauthorized
2 Date: Wed, 07 Oct 2026 17:32:32 GMT
3 Content-Type: application/json
4 Content-Length: 51
5 X-Request-Id: 3af531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Token tidak ditemukan"
11 }
```

**Gambar 3.16** *Pengujian 13: Daftar Mata Kuliah tanpa Token (401)*

### 3.4 Pengujian Data Mahasiswa

Daftar mahasiswa tanpa parameter mengembalikan 10 data pertama. Di bagian akhir response, objek meta menunjukkan current\_page 1, per\_page 10, total 20, dan last\_page 2.

```
● Response(56ms)
1 HTTP/1.1 200 OK
2 Date: Wed, 07 Oct 2026 17:33:00 GMT
3 Content-Type: application/json
4 Content-Length: 2809
5 X-Request-Id: 3bf531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Data mahasiswa berhasil diambil",
11  "data": [
12    {
13      "id": 1,
14      "user_id": 2,
15      "nim": "187221000001",
16      "nama": "Rina Putri",
17      "email": "187221000001@student.siakad.ac.id",
18      "prodi": "Sistem Informasi",
19      "angkatan": 2022,
20      "ipk_terakhir": 3.45,
21      "created_at": "2026-10-07T22:57:06.617593+07:00",
22      "updated_at": "2026-10-07T22:57:06.617593+07:00"
23    },
24    {
25      "id": 2,
26      "user_id": 3,
27      "nim": "187221000002",
28      "nama": "Budi Santoso",
29      "email": "187221000002@student.siakad.ac.id",
30      "prodi": "Sistem Informasi",
31      "angkatan": 2022,
32      "ipk_terakhir": 2.75,
33      "created_at": "2026-10-07T22:57:06.617593+07:00",
34      "updated_at": "2026-10-07T22:57:06.617593+07:00"
35    },
36    {
37      "id": 3,
38      "user_id": 4,
39      "nim": "187221000003",
40      "nama": "Siti Aminah",
41      "email": "187221000003@student.siakad.ac.id",
42      "prodi": "Sistem Informasi",
43      "angkatan": 2022,
44      "ipk_terakhir": 3.8,
```

**Gambar 3.17** *Pengujian 14: Daftar Mahasiswa Bagian Awal (200)*

```

● Response(56ms)
117     "created_at": "2026-10-07T22:57:06.617593+07:00",
118     "updated_at": "2026-10-07T22:57:06.617593+07:00"
119 },
120 {
121     "id": 10,
122     "user_id": 11,
123     "nim": "187221000010",
124     "nama": "Jihan Aulia",
125     "email": "187221000010@student.siakad.ac.id",
126     "prodi": "Teknik Informatika",
127     "angkatan": 2023,
128     "ipk_terakhir": 2.1,
129     "created_at": "2026-10-07T22:57:06.617593+07:00",
130     "updated_at": "2026-10-07T22:57:06.617593+07:00"
131 }
132 ],
133 "meta": {
134     "current_page": 1,
135     "per_page": 10,
136     "total": 20,
137     "last_page": 2
138 }
139 }

```

**Gambar 3.18** Pengujian 14: Objek meta pada Daftar Mahasiswa

Dengan page=2, per\_page=5, dan sort=-ipk\_terakhir, data diurutkan dari IPK tertinggi, dan yang tampil adalah urutan keenam sampai kesepuluh. Objek meta-nya pun berubah jadi per\_page 5 dan last\_page 4.

```
● Response(15ms)
1 HTTP/1.1 200 OK
2 Date: Wed, 07 Oct 2026 17:33:57 GMT
3 Content-Type: application/json
4 Content-Length: 1471
5 X-Request-Id: 3cf531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Data mahasiswa berhasil diambil",
11  "data": [
12    {
13      "id": 1,
14      "user_id": 2,
15      "nim": "187221000001",
16      "nama": "Rina Putri",
17      "email": "187221000001@student.siakad.ac.id",
18      "prodi": "Sistem Informasi",
19      "angkatan": 2022,
20      "ipk_terakhir": 3.45,
21      "created_at": "2026-10-07T22:57:06.617593+07:00",
22      "updated_at": "2026-10-07T22:57:06.617593+07:00"
23    },
24    {
25      "id": 9,
26      "user_id": 10,
27      "nim": "187221000009",
28      "nama": "Irfan Maulana",
29      "email": "187221000009@student.siakad.ac.id",
30      "prodi": "Teknik Informatika",
31      "angkatan": 2022,
32      "ipk_terakhir": 3.25,
33      "created_at": "2026-10-07T22:57:06.617593+07:00",
34      "updated_at": "2026-10-07T22:57:06.617593+07:00"
35    },
36    {
37      "id": 17,
38      "user_id": 18,
39      "nim": "187221000017",
40      "nama": "Raka Firmansyah",
41      "email": "187221000017@student.siakad.ac.id",
42      "prodi": "Sains Data",
43      "angkatan": 2024,
44      "ipk_terakhir": 3.15,
```

**Gambar 3.19** *Pengujian 15: Pagination dan Urutan IPK Tertinggi (200)*

```
● Response(15ms)
57     "created_at": "2026-10-07T22:57:06.617593+07:00",
58     "updated_at": "2026-10-07T22:57:06.617593+07:00"
59 },
60 {
61     "id": 13,
62     "user_id": 14,
63     "nim": "187221000013",
64     "nama": "Muhammad Rizki",
65     "email": "187221000013@student.siakad.ac.id",
66     "prodi": "Teknik Informatika",
67     "angkatan": 2024,
68     "ipk_terakhir": 3,
69     "created_at": "2026-10-07T22:57:06.617593+07:00",
70     "updated_at": "2026-10-07T22:57:06.617593+07:00"
71 }
72 ],
73 "meta": {
74     "current_page": 2,
75     "per_page": 5,
76     "total": 20,
77     "last_page": 4
78 }
79 }
```

**Gambar 3.20** Pengujian 15: Objek meta dengan per\_page 5

```
● Response(41ms)
1 HTTP/1.1 200 OK
2 Date: Wed, 07 Oct 2026 17:34:35 GMT
3 Content-Type: application/json
4 Content-Length: 398
5 X-Request-Id: 3df531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Data mahasiswa berhasil diambil",
11  "data": [
12    {
13      "id": 1,
14      "user_id": 2,
15      "nim": "187221000001",
16      "nama": "Rina Putri",
17      "email": "187221000001@student.siakad.ac.id",
18      "prodi": "Sistem Informasi",
19      "angkatan": 2022,
20      "ipk_terakhir": 3.45,
21      "created_at": "2026-10-07T22:57:06.617593+07:00",
22      "updated_at": "2026-10-07T22:57:06.617593+07:00"
23    }
24  ],
25  "meta": {
26    "current_page": 1,
27    "per_page": 10,
28    "total": 1,
29    "last_page": 1
30  }
31 }
```

**Gambar 3.21** *Pengujian 16: Filter Prodi, Angkatan, dan Pencarian (200)*

Daftar mahasiswa hanya boleh diakses admin, jadi token mahasiswa dibalas 403.

```
● Response(44ms)
1 HTTP/1.1 403 Forbidden
2 Date: Wed, 07 Oct 2026 17:35:02 GMT
3 Content-Type: application/json
4 Content-Length: 71
5 X-Request-Id: 3ef531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Anda tidak memiliki akses ke resource ini"
11 }
```

**Gambar 3.22** *Pengujian 17: Daftar Mahasiswa dengan Token Mahasiswa (403)*

Penambahan mahasiswa baru menghasilkan 201 Created dengan id 21 dan user\_id 22. Akun ini langsung bisa dipakai login dengan password awal yang sama dengan NIM-nya.

```
● Response(27ms)
1 HTTP/1.1 201 Created
2 Date: Wed, 07 Oct 2026 17:35:32 GMT
3 Content-Type: application/json
4 Content-Length: 327
5 X-Request-Id: 3ff531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Mahasiswa berhasil ditambahkan",
11  "data": {
12    "id": 21,
13    "user_id": 22,
14    "nim": "187221000099",
15    "nama": "Zahra Amelia",
16    "email": "zahra@student.siakad.ac.id",
17    "prodi": "Sistem Informasi",
18    "angkatan": 2024,
19    "ipk_terakhir": 3.2,
20    "created_at": "2026-10-08T00:35:32.296997+07:00",
21    "updated_at": "2026-10-08T00:35:32.296997+07:00"
22  }
23 }
```

**Gambar 3.23** *Pengujian 18: Tambah Mahasiswa (201)*

Saat data dengan NIM dan email yang sama dikirim ulang, server membalas 422 dengan dua pesan sekaligus, yaitu "NIM sudah terdaftar" dan "Email sudah terdaftar". Data yang tidak valid juga dibalas 422, dengan pesan untuk setiap field yang bermasalah.

```
● Response(39ms)
1 HTTP/1.1 422 Unprocessable Entity
2 Date: Wed, 07 Oct 2026 17:36:24 GMT
3 Content-Type: application/json
4 Content-Length: 119
5 X-Request-Id: 40f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Validasi gagal",
11  "errors": {
12    "email": [
13      "Email sudah terdaftar"
14    ],
15    "nim": [
16      "NIM sudah terdaftar"
17    ]
18  }
19 }
```

**Gambar 3.24** *Pengujian 19: Tambah Mahasiswa dengan NIM dan Email Ganda (422)*

```
● Response(8ms)
1 HTTP/1.1 422 Unprocessable Entity
2 Date: Wed, 07 Oct 2026 17:37:15 GMT
3 Content-Type: application/json
4 Content-Length: 320
5 X-Request-Id: 41f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Validasi gagal",
11  "errors": {
12    "angkatan": [
13      "Angkatan tidak boleh melebihi tahun berjalan"
14    ],
15    "email": [
16      "Format email tidak valid"
17    ],
18    "ipk_terakhir": [
19      "IPK terakhir harus di antara 0.00 dan 4.00"
20    ],
21    "nama": [
22      "Nama wajib diisi"
23    ],
24    "nim": [
25      "NIM harus terdiri dari 12 digit angka"
26    ],
27    "prodi": [
28      "Prodi wajib diisi"
29    ]
30  }
31 }
```

**Gambar 3.25** Pengujian 20: Tambah Mahasiswa dengan Data Tidak Valid (422)

```
● Response(7ms)
1 HTTP/1.1 200 OK
2 Date: Wed, 07 Oct 2026 17:37:44 GMT
3 Content-Type: application/json
4 Content-Length: 380
5 X-Request-Id: 42f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Detail mahasiswa berhasil diambil",
11  "data": {
12    "id": 1,
13    "user_id": 2,
14    "nim": "187221000001",
15    "nama": "Rina Putri",
16    "email": "187221000001@student.siakad.ac.id",
17    "prodi": "Sistem Informasi",
18    "angkatan": 2022,
19    "ipk_terakhir": 3.45,
20    "created_at": "2026-10-07T22:57:06.617593+07:00",
21    "updated_at": "2026-10-07T22:57:06.617593+07:00",
22    "mata_kuliah": [],
23    "total_sks": 0,
24    "batas_sks": 24
25  }
26 }
```

**Gambar 3.26** Pengujian 21: Detail Mahasiswa oleh Admin (200)

Mahasiswa bisa melihat datanya sendiri (id 1), tapi ditolak 403 saat mencoba membuka data mahasiswa lain (id 2). Sementara itu, id yang tidak ada dibalas 404.

```

● Response(17ms)
1 HTTP/1.1 200 OK
2 Date: Wed, 07 Oct 2026 17:38:24 GMT
3 Content-Type: application/json
4 Content-Length: 380
5 X-Request-Id: 43f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Detail mahasiswa berhasil diambil",
11  "data": {
12    "id": 1,
13    "user_id": 2,
14    "nim": "187221000001",
15    "nama": "Rina Putri",
16    "email": "187221000001@student.siakad.ac.id",
17    "prodi": "Sistem Informasi",
18    "angkatan": 2022,
19    "ipk_terakhir": 3.45,
20    "created_at": "2026-10-07T22:57:06.617593+07:00",
21    "updated_at": "2026-10-07T22:57:06.617593+07:00",
22    "mata_kuliah": [],
23    "total_sks": 0,
24    "batas_sks": 24
25  }
26 }

```

**Gambar 3.27** Pengujian 22: Mahasiswa Melihat Data Sendiri (200)

```

● Response(47ms)
1 HTTP/1.1 403 Forbidden
2 Date: Wed, 07 Oct 2026 17:38:54 GMT
3 Content-Type: application/json
4 Content-Length: 76
5 X-Request-Id: 44f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Anda tidak boleh mengakses data mahasiswa lain"
11 }

```

**Gambar 3.28** Pengujian 23: Mahasiswa Melihat Data Orang Lain (403)

```
● Response(31ms)
1 HTTP/1.1 404 Not Found
2 Date: Wed, 07 Oct 2026 17:39:54 GMT
3 Content-Type: application/json
4 Content-Length: 55
5 X-Request-Id: 45f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Mahasiswa tidak ditemukan"
11 }
```

**Gambar 3.29** *Pengujian 24: Detail Mahasiswa yang Tidak Ada (404)*

```
● Response(24ms)
1 HTTP/1.1 200 OK
2 Date: Wed, 07 Oct 2026 17:40:46 GMT
3 Content-Type: application/json
4 Content-Length: 335
5 X-Request-Id: 46f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": true,
10  "message": "Data mahasiswa berhasil diperbarui",
11  "data": {
12    "id": 1,
13    "user_id": 2,
14    "nim": "187221000001",
15    "nama": "Rina Putri",
16    "email": "187221000001@student.siakad.ac.id",
17    "prodi": "Sistem Informasi",
18    "angkatan": 2022,
19    "ipk_terakhir": 3.45,
20    "created_at": "2026-10-07T22:57:06.617593+07:00",
21    "updated_at": "2026-10-08T00:40:46.914983+07:00"
22  }
23 }
```

**Gambar 3.30** *Pengujian 25: Ubah Data Mahasiswa (200)*

```
● Response(41ms)
1 HTTP/1.1 422 Unprocessable Entity
2 Date: Wed, 07 Oct 2026 17:41:14 GMT
3 Content-Type: application/json
4 Content-Length: 88
5 X-Request-Id: 47f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Validasi gagal",
11  "errors": {
12    "nim": [
13      "NIM tidak boleh diubah"
14    ]
15  }
16 }
```

**Gambar 3.31** *Pengujian 26: Mencoba Mengubah NIM (422)*

Soft delete untuk mahasiswa id 20 menghasilkan 204 No Content tanpa isi response. Saat penghapusan yang sama diulang, hasilnya 404, karena data itu sudah dianggap tidak ada.

```
● Response(27ms)
1 HTTP/1.1 204 No Content
2 Date: Wed, 07 Oct 2026 17:42:15 GMT
3 X-Request-Id: 48f531c8-8e47-48c9-93d6-4cfb24e64d64
4 Connection: close
5
```

**Gambar 3.32** *Pengujian 27: Soft Delete Mahasiswa (204)*

```
● Response(19ms)
1 HTTP/1.1 404 Not Found
2 Date: Wed, 07 Oct 2026 17:42:47 GMT
3 Content-Type: application/json
4 Content-Length: 55
5 X-Request-Id: 49f531c8-8e47-48c9-93d6-4cfb24e64d64
6 Connection: close
7
8 {
9   "success": false,
10  "message": "Mahasiswa tidak ditemukan"
11 }
```

**Gambar 3.33** Pengujian 28: Menghapus Data yang Sudah Dihapus (404)

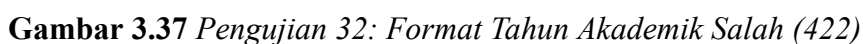
### 3.5 Pengujian Kartu Rencana Studi

Di sini Rina Putri (IPK 3,45) mengambil MK001 Pemrograman Backend Lanjut sebanyak 4 SKS. Response 201 Created menampilkan total\_sks 4, batas\_sks 24, dan sisa\_sks 20, sesuai business rule 1 untuk IPK di atas 3,00.

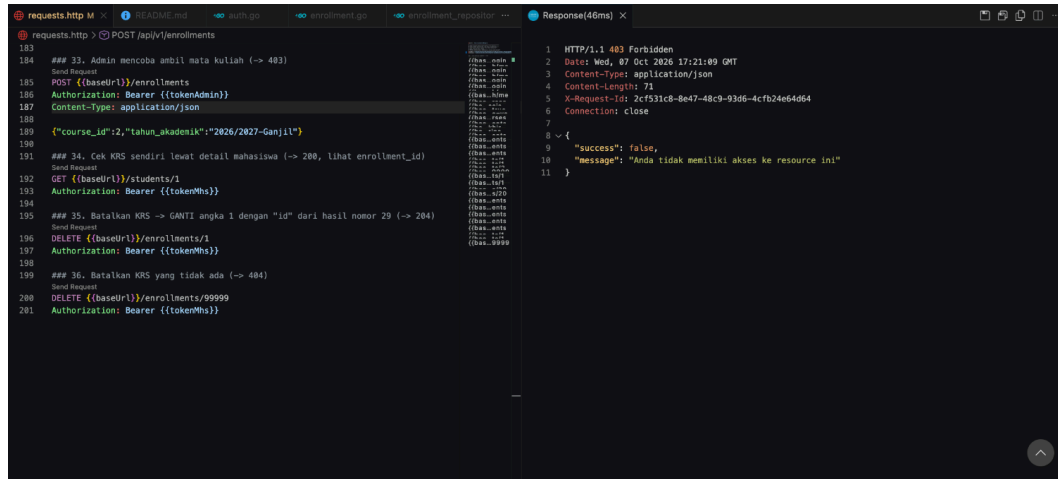
```
requests.http M x README.md auth.go enrollment.go enrollment_repository ... Response(113ms) X
requests.http M x POST /api/v1/enrollments
154 ### 29. Ambil mata kuliah MK001 (-> 201)
155
156 ## Request
157 POST {{baseurl}}/enrollments
158 Authorization: Bearer {{tokenMhs}}
159 Content-Type: application/json
160
161 {"course_id":1,"tahun_akademik":"2026/2027-Ganjil"}
162
163 ## 30. Ambil MK001 lagi di tahun yang sama (-> 409)
164
165 ## Request
166 POST {{baseurl}}/enrollments
167 Authorization: Bearer {{tokenMhs}}
168 Content-Type: application/json
169
170 {"course_id":1,"tahun_akademik":"2026/2027-Ganjil"}
171
172 ## 31. Mata kuliah tidak ada (-> 422)
173
174 ## Request
175 POST {{baseurl}}/enrollments
176 Authorization: Bearer {{tokenMhs}}
177 Content-Type: application/json
178
179 {"course_id":999,"tahun_akademik":"2026/2027-Ganjil"}
180
181 ## 32. Format tahun akademik salah (-> 422)
182
183 ## Request
184 POST {{baseurl}}/enrollments
185 Authorization: Bearer {{tokenMhs}}
186 Content-Type: application/json
187
188 {"course_id":2,"tahun_akademik":"2026-Ganjil"}
189
190 ## 33. Admin mencoba ambil mata kuliah (-> 403)
191
192 ## Request
193 POST {{baseurl}}/enrollments
194 Authorization: Bearer {{tokenAdmin}}
195 Content-Type: application/json
196
197 {"course_id":1,"tahun_akademik":"2026/2027-Ganjil"}
198
199 ## Response
200
201 HTTP/1.1 201 Created
202 Date: Wed, 07 Oct 2026 17:17:24 GMT
203 Content-Type: application/json
204 Content-Length: 305
205 X-Request-Id: 28f531c8-8e47-48c9-93d6-4cfb24e64d64
206 Connection: close
207
208 {
209   "success": true,
210   "message": "Mata kuliah berhasil ditambahkan ke KRS",
211   "data": {
212     "id": 1,
213     "student_id": 1,
214     "course_id": 1,
215     "kode_mk": "MK001",
216     "nama_mk": "Pemrograman Backend Lanjut",
217     "sks": 4,
218     "tahun_akademik": "2026/2027-Ganjil",
219     "created_at": "2026-10-08T08:17:24.791998+07:00",
220     "total_sks": 4,
221     "batas_sks": 24,
222     "sisa_sks": 20
223   }
224 }
```

**Gambar 3.34** Pengujian 29: Mengambil Mata Kuliah (201)

Saat MK001 diambil lagi di tahun akademik yang sama, server menolaknya dengan 409 Conflict. Ini membuktikan business rule 2 sudah berjalan.

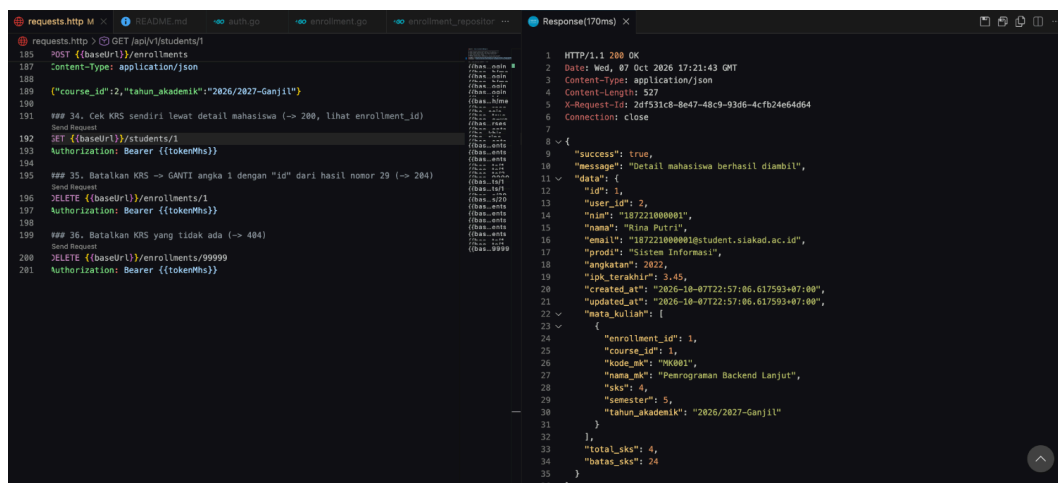


Admin yang mencoba mengambil mata kuliah ditolak 403 oleh middleware RequireRole, karena endpoint KRS memang khusus untuk mahasiswa.

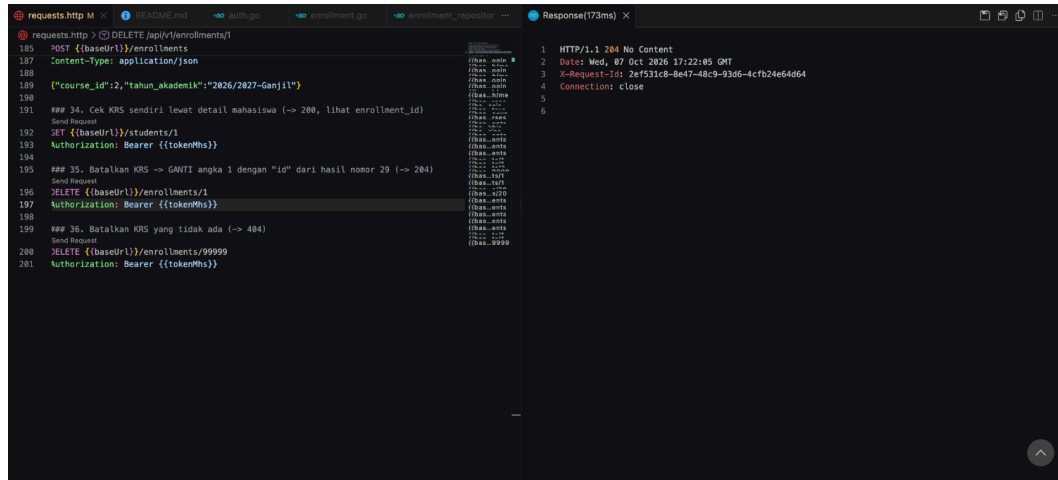


**Gambar 3.38** *Pengujian 33: Admin Mencoba Mengambil Mata Kuliah (403)*

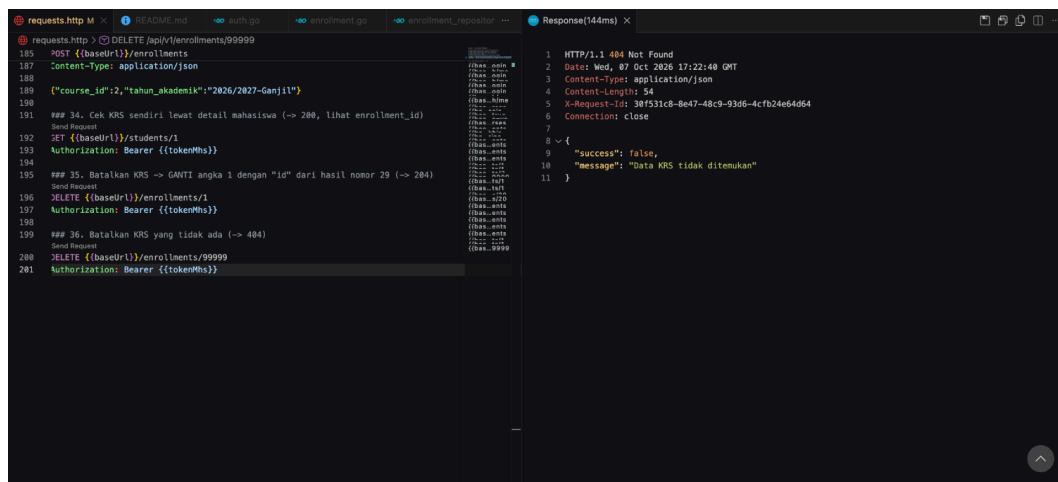
Sekarang detail mahasiswa menampilkan MK001 di dalam mata\_kuliah, lengkap dengan enrollment\_id 1. Id inilah yang kemudian saya pakai untuk membatalkan KRS.



**Gambar 3.39** *Pengujian 34: KRS Terlihat pada Detail Mahasiswa (200)*



**Gambar 3.40** *Pengujian 35: Membatalkan Mata Kuliah dari KRS (204)*



**Gambar 3.41** *Pengujian 36: Membatalkan KRS yang Tidak Ada (404)*

### 3.6 Rekapitulasi Hasil Pengujian

Semua hasil pengujian saya rangkum di Tabel 3.2. Dari 36 pengujian, semuanya menghasilkan status code yang sesuai harapan. Status code yang wajib dipakai menurut soal (200, 201, 204, 401, 403, 404, 409, 422, 429, dan 500) juga sudah diterapkan semua. Untuk status 429, buktinya terlihat dari header X-Ratelimit di Pengujian 5, sedangkan status 500 ditangani oleh ErrorHandler terpusat tanpa membocorkan stack trace.

**Tabel 3.2** *Rekapitulasi Hasil Pengujian Endpoint*

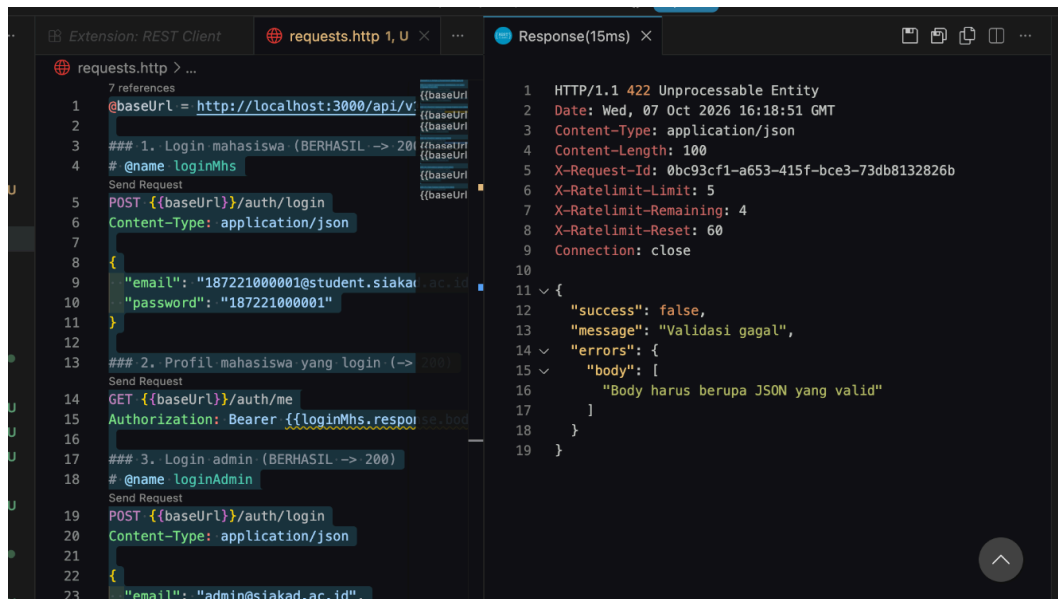
| No | Skenario                        | Diharapkan | Hasil |
|----|---------------------------------|------------|-------|
| 1  | Login mahasiswa                 | 200        | 200   |
| 2  | Profil mahasiswa                | 200        | 200   |
| 3  | Login admin                     | 200        | 200   |
| 4  | Profil admin                    | 200        | 200   |
| 5  | Password salah                  | 401        | 401   |
| 6  | Validasi login gagal            | 422        | 422   |
| 7  | Profil tanpa token              | 401        | 401   |
| 8  | Profil dengan token palsu       | 401        | 401   |
| 9  | Daftar mata kuliah              | 200        | 200   |
| 10 | Filter semester dan pencarian   | 200        | 200   |
| 11 | Mata kuliah belum penuh         | 200        | 200   |
| 12 | Query courses tidak valid       | 422        | 422   |
| 13 | Courses tanpa token             | 401        | 401   |
| 14 | Daftar mahasiswa                | 200        | 200   |
| 15 | Pagination dan sort IPK         | 200        | 200   |
| 16 | Filter prodi, angkatan, search  | 200        | 200   |
| 17 | Daftar mahasiswa oleh mahasiswa | 403        | 403   |
| 18 | Tambah mahasiswa                | 201        | 201   |
| 19 | NIM dan email ganda             | 422        | 422   |
| 20 | Data mahasiswa tidak valid      | 422        | 422   |
| 21 | Detail oleh admin               | 200        | 200   |
| 22 | Detail data sendiri             | 200        | 200   |
| 23 | Detail data orang lain          | 403        | 403   |
| 24 | Detail id tidak ada             | 404        | 404   |
| 25 | Ubah data mahasiswa             | 200        | 200   |
| 26 | Mengubah NIM                    | 422        | 422   |
| 27 | Soft delete mahasiswa           | 204        | 204   |
| 28 | Hapus ulang data terhapus       | 404        | 404   |
| 29 | Ambil mata kuliah               | 201        | 201   |
| 30 | Ambil mata kuliah dua kali      | 409        | 409   |
| 31 | Mata kuliah tidak ditemukan     | 422        | 422   |
| 32 | Format tahun akademik salah     | 422        | 422   |
| 33 | Admin mengambil mata kuliah     | 403        | 403   |
| 34 | KRS pada detail mahasiswa       | 200        | 200   |
| 35 | Batalkan KRS                    | 204        | 204   |
| 36 | Batalkan KRS yang tidak ada     | 404        | 404   |

## BAB IV KENDALA SELAMA Pengerjaan

Di bab ini saya ceritakan kendala yang benar-benar saya alami selama pengerjaan, beserta cara mengatasinya. Jadi ini bukan versi yang sudah dirapikan seolah-olah semuanya langsung berhasil, tapi proses debugging yang sebenarnya.

### 4.1 Request Login dari REST Client Selalu Dibalas 422

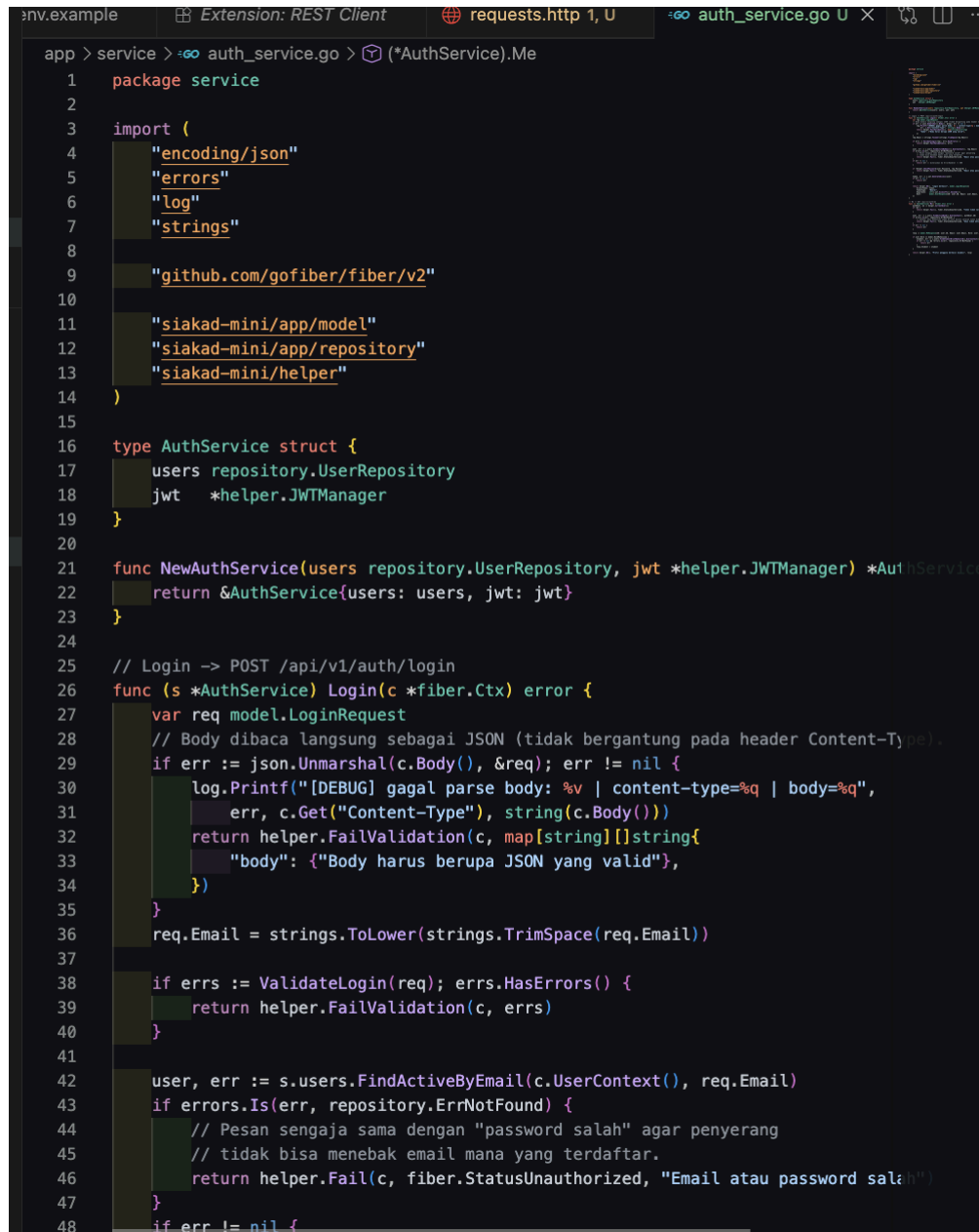
Waktu pertama kali menguji endpoint login pakai REST Client, request yang isinya sudah benar justru selalu dibalas **422 Unprocessable Entity** dengan pesan "Body harus berupa JSON yang valid". Padahal email dan password yang saya kirim adalah akun hasil seeder.



**Gambar 4.1** Request Login Benar tetapi Dibalas 422

Langkah pertama yang saya lakukan adalah memisahkan sumber masalahnya. Request yang sama saya kirim lewat curl di terminal, dan hasilnya **200 Login berhasil**. Dari sini jelas bahwa kode server sudah benar, dan masalahnya ada di cara REST Client mengirim request. Awalnya saya menduga penyebabnya karakter spasi tak terlihat (non-breaking space) dari hasil copy-paste, jadi JSON-nya saya tulis ulang dalam satu baris. Tapi hasilnya tetap 422.

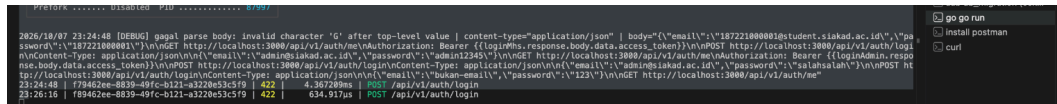
Supaya tidak menebak-nebak, saya tambahkan satu baris log sementara di fungsi Login. Kalau body gagal dibaca, server akan mencetak isi header Content-Type dan isi body ke terminal. Sekalian, cara membaca body saya ubah dari `c.BodyParser` menjadi `json.Unmarshal(c.Body(), &req)`, supaya tidak bergantung pada header Content-Type.



```
app > service > auth_service.go > (*AuthService).Me
1 package service
2
3 import (
4     "encoding/json"
5     "errors"
6     "log"
7     "strings"
8
9     "github.com/gofiber/fiber/v2"
10
11     "siakad-mini/app/model"
12     "siakad-mini/app/repository"
13     "siakad-mini/helper"
14 )
15
16 type AuthService struct {
17     users repository.UserRepository
18     jwt *helper.JWTManager
19 }
20
21 func NewAuthService(users repository.UserRepository, jwt *helper.JWTManager) *AuthService {
22     return &AuthService{users: users, jwt: jwt}
23 }
24
25 // Login -> POST /api/v1/auth/login
26 func (s *AuthService) Login(c *fiber.Ctx) error {
27     var req model.LoginRequest
28     // Body dibaca langsung sebagai JSON (tidak bergantung pada header Content-Type).
29     if err := json.Unmarshal(c.Body(), &req); err != nil {
30         log.Printf("[DEBUG] gagal parse body: %v | content-type=%q | body=%q",
31             err, c.Get("Content-Type"), string(c.Body()))
32         return helper.FailValidation(c, map[string][]string{
33             "body": {"Body harus berupa JSON yang valid"},
34         })
35     }
36     req.Email = strings.ToLower(strings.TrimSpace(req.Email))
37
38     if errs := ValidateLogin(req); errs.HasErrors() {
39         return helper.FailValidation(c, errs)
40     }
41
42     user, err := s.users.FindActiveByEmail(c.UserContext(), req.Email)
43     if errors.Is(err, repository.ErrNotFound) {
44         // Pesan sengaja sama dengan "password salah" agar penyerang
45         // tidak bisa menebak email mana yang terdaftar.
46         return helper.Fail(c, fiber.StatusUnauthorized, "Email atau password salah")
47     }
48     if err != nil {
```

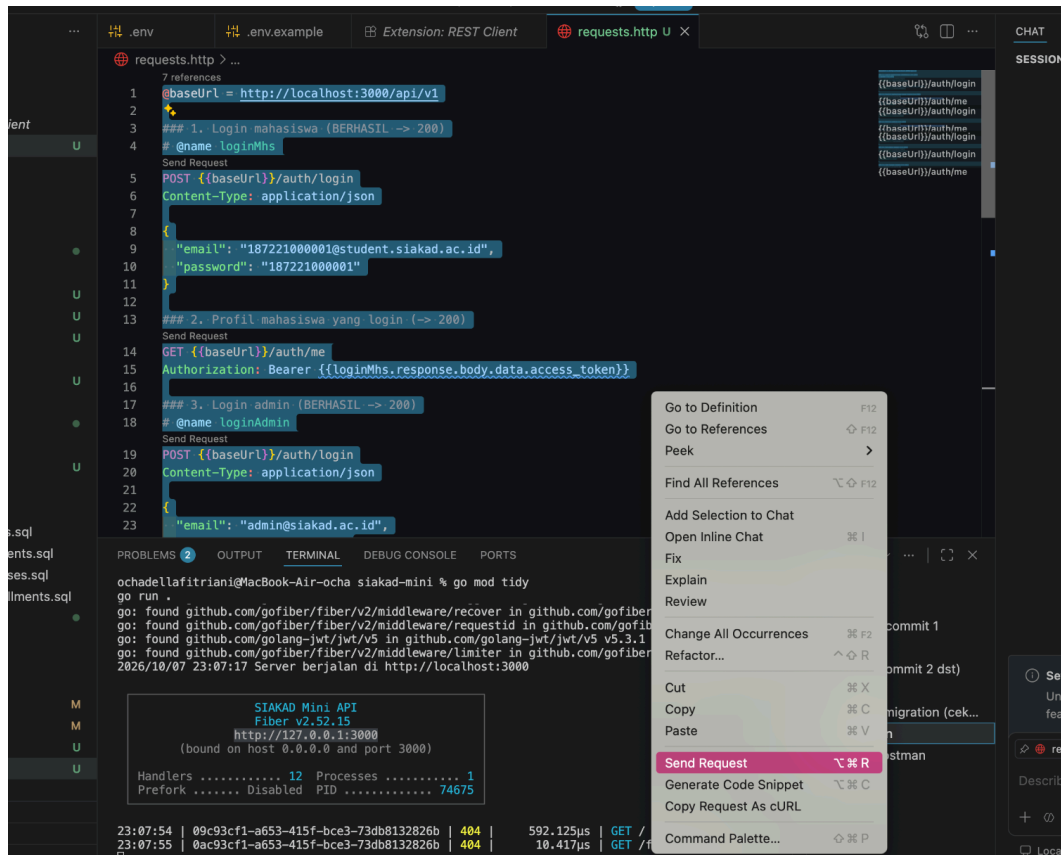
**Gambar 4.2** Log Debug Sementara pada Fungsi Login

Dari log itu, penyebabnya langsung kelihatan. Body yang diterima server ternyata bukan cuma JSON login, tapi **seluruh isi file requests.http** dari request nomor 1 sampai 7. Karena itu json.Unmarshal gagal dengan pesan invalid character 'G' after top-level value, di mana huruf G itu berasal dari baris GET milik request berikutnya.



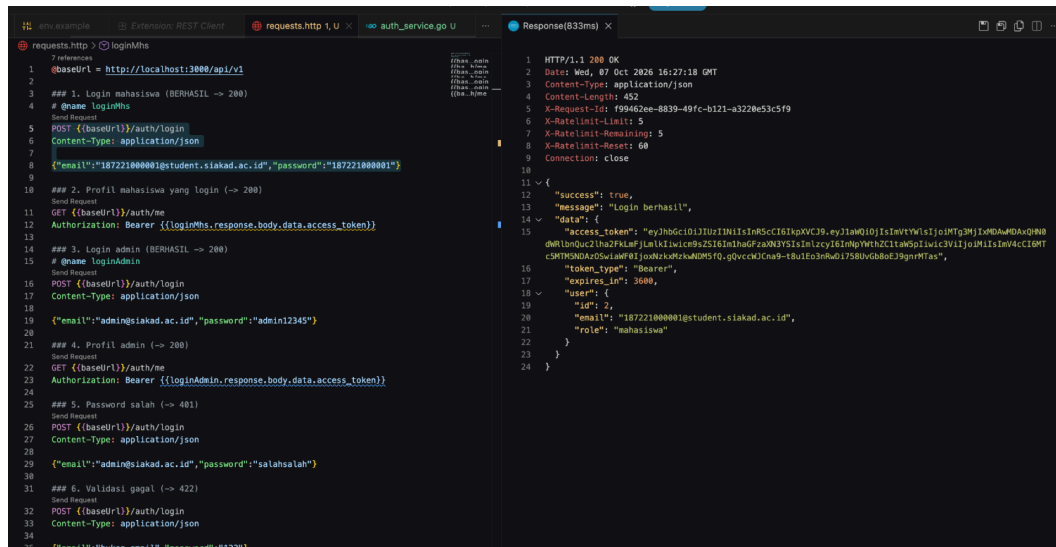
**Gambar 4.3** Isi Body yang Diterima Server Ternyata Seluruh Isi File

Ternyata penyebabnya sepele: seluruh teks di file requests.http masih dalam keadaan terseleksi (terblok) setelah saya menekan Cmd+A waktu menempel isi file. REST Client punya aturan begini: kalau ada teks yang terseleksi, yang dikirim adalah teks yang terseleksi itu, bukan request di bawah tulisan Send Request yang diklik. Status bar VS Code sebenarnya juga sudah memberi petunjuk, yaitu tulisan seperti "1035 selected".



**Gambar 4.4** *Seluruh Isi File Terseleksi saat Send Request Dijalankan*

Cara memperbaikinya, seleksinya saya hilangkan dulu dengan menekan tombol panah atau mengklik baris kosong. Setelah itu saya klik langsung tulisan Send Request, tanpa memblok teks dan tanpa klik kanan. Hasilnya, request login yang sama dibalas **200 Login berhasil**. Baris log debug-nya kemudian saya hapus sebelum commit, karena log itu ikut mencatat isi body, termasuk password, ke terminal.



Setelah itu muncul kendala lanjutan di request `/auth/me` yang memakai variabel otomatis `{{loginMhs.response.body.data.access_token}}`. Request ini terus dibalas **401 Token tidak valid**, karena variabelnya tidak terbaca dan REST Client mengirim teks variabel itu apa adanya sebagai token. Supaya pengujian berikutnya tidak bergantung pada fitur yang kurang stabil ini, cara pengisian token saya ganti menjadi variabel manual `@tokenMhs` dan `@tokenAdmin` di bagian atas file, seperti di Gambar 3.3. Dengan cara ini, semua request yang butuh token bisa berjalan dengan lancar.

**Gambar 4.6** Variabel Token Otomatis Tidak Terbaca sehingga Dibalas 401

## **BAB V PENUTUP**

### **5.1 Kesimpulan**

Lewat UTS ini, saya berhasil membangun RESTful API SIAKAD Mini dari folder kosong menggunakan Go, Fiber v2, dan PostgreSQL, dengan struktur folder yang mengikuti Clean Architecture dari Modul Pertemuan 4. Kesepuluh endpoint yang diminta sudah berjalan dan sudah saya uji lewat 36 skenario pengujian, dan semuanya menghasilkan status code yang sesuai rancangan.

Semua ketentuan wajib UTS juga sudah terpenuhi, yaitu migration empat tabel dengan relasi dan constraint yang sesuai model data, seeder berisi 1 admin, 20 mahasiswa, dan 10 mata kuliah, password yang disimpan dalam bentuk hash bcrypt, autentikasi token JWT di semua endpoint kecuali login, serta riwayat commit bertahap di GitHub. Format response JSON-nya pun seragam, karena semuanya dibentuk lewat satu helper yang sama.

Keempat business rule juga sudah dijalankan secara konsisten. Batas SKS cukup ditulis sekali sebagai fungsi `BatasSKS` dan dipakai ulang oleh dua endpoint. Larangan mengambil mata kuliah yang sama dua kali dijaga di aplikasi (409) sekaligus di database lewat constraint `UNIQUE`. Pengecekan kuota dijalankan di dalam transaction dengan row locking `SELECT ... FOR UPDATE`, supaya kuota tidak terlampaui walaupun banyak mahasiswa mendaftar bersamaan. Sementara itu, pembatasan akses ke data milik sendiri dijaga di layer service, karena keputusannya bergantung pada isi data dan tidak bisa diputuskan oleh middleware.

Dari pengerjaan ini, kendala terbesar yang saya alami justru bukan dari kode, tapi dari alat pengujiannya. Pelajaran yang saya ambil: kalau ada masalah, sebaiknya pisahkan dulu sumbernya (di server atau di alat uji), lalu tambahkan log sementara untuk melihat data yang benar-benar diterima server, daripada menebak-nebak penyebabnya.

## **5.2 Tautan Repositori GitHub**

Seluruh kode sumber, file migration, seeder, README, dan file pengujian requests.http bisa dilihat di repositori berikut.

**LINK GITHUB :**

**Repositori UTS:** [https://github.com/ochadella/siakad-mini\\_UTSPBL](https://github.com/ochadella/siakad-mini_UTSPBL)

**Repositori Latihan Fiber (folder UTS):**

[https://github.com/ochadella/PBL\\_Latihanfiber/tree/main/UTS](https://github.com/ochadella/PBL_Latihanfiber/tree/main/UTS)